TrekIndex: A Single Index for d-Separation-Based Queries on Large Causal DAGs

Huizhong Wang
Lyon 1 University, CNRS Liris
Lyon, France
huizhong.wang@univ-lyon1.fr

Angela Bonifati
Lyon 1 University, CNRS Liris
Lyon, France
angela.bonifati@univ-lyon1.fr

ABSTRACT

d-Separation is the graphical criterion that decides which conditional independences a causal DAG implies, and it underpins many causal analysis tasks. Testing a DAG against data and benchmarking causal discovery on a known DAG issue many d-separation queries on the same DAG. As causal DAGs grow to millions of variables, existing algorithms, which traverse the DAG for every query, become inefficient, and adapting reachability indexes is non-trivial: conditioning can open paths as well as block them, separation requires every path to be blocked, and criteria such as the back-door criterion test d-separation while ignoring edges that depend on the query. To address these issues, we present TrekIndex, the first index for d-separation with arbitrary conditioning sets and ignored edges. Its key idea is to separate the two roles of conditioning: blocking becomes forbidden states on a trek DAG that does not depend on the query, while a small graph explored at query time captures the paths that conditioning opens, so every query reduces to forbidden-set reachability on one DAG. TrekIndex decides most of these tests without a search, since 2-hop labels refute or certify them, and witness contraction keeps the labels small. Ignored edges likewise forbid states, so the same index answers the criteria built on d-separation. On six datasets with up to 12.2 million vertices, TrekIndex answers d-separation queries 4.84× to 6,935× faster than the fastest of six traversal baselines, queries of eight criteria 69.6× to 5,552× faster, and three application workloads 1.42× to 4,696× faster.

PVLDB Reference Format:
Huizhong Wang and Angela Bonifati. TrekIndex: A Single Index for d-Separation-Based Queries on Large Causal DAGs. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/HuiZ-W/TrekIndex.

1 INTRODUCTION

Causal analysis seeks to discover, represent, and measure cause–effect relationships. It informs decisions in fields such as medicine, economics, and public policy [26], and it has become a major theme in data management research, e.g., explaining query answers [19, 20, 32], removing bias from OLAP queries [33], learning causal models from relational data [34], what-if analysis [9, 22], and causal DAG summarization [54]. Pearl's framework of causality [24, 26] represents causal knowledge as a causal directed acyclic graph, or *causal DAG* $\mathcal{G}$ (Figure 1): each vertex is a causal variable, and each edge $u \to v$ states that u is a direct cause of v.

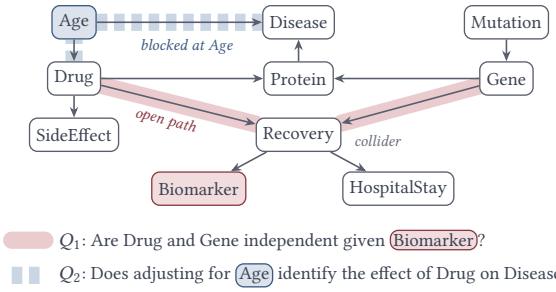


Figure 1: A causal DAG with two queries.

Since both statistical dependence and causal influence propagate along the paths of a causal DAG, many causal questions can be formulated as queries on these paths. The most fundamental of these queries is *d-separation* [10, 26], which decides whether the DAG implies that two sets of variables are independent given a third. Many causal analysis tasks are built on d-separation and issue many d-separation queries on the same DAG. For example, testing a DAG against data checks the conditional independences that the DAG implies [41], and constraint-based causal discovery, such as the PC algorithm [8, 39], learns the equivalence class of a causal DAG from conditional-independence tests with conditioning sets of increasing size. When such tasks are evaluated on a known DAG, each test is a d-separation query.

Given disjoint sets of variables X and Y and a *conditioning set* Z of observed variables, a path between X and Y is *open* if it can transmit association once Z is observed. X and Y are *d-separated* by Z if no path between them is open, and *d-connected* otherwise. Conditioning on a vertex of a path blocks the path, unless the vertex is a *collider*, at which both edges of the path meet head to head. A collider, in contrast, blocks the path unless it or one of its descendants is conditioned on. In Figure 1, conditioning on Age blocks the path Drug ← Age → Disease (blue). The path Drug → Recovery ← Gene (red), however, is blocked by the collider Recovery when nothing is observed, and conditioning on Biomarker, a descendant of Recovery, opens it. Drug and Gene are thus d-connected given Biomarker (Q_1), although neither causes the other.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Existing algorithms answer each d-separation query by traversing the DAG. The linear-time search of Geiger et al. [10] and Bayes-Ball [35] follow the paths from X until they are blocked. Other algorithms restrict this search to the ancestors of the query variables [17] or to relevant vertices [5], and CIfly [49] compiles d-separation into rule tables that drive the same kind of search. In graph databases, the d-collision graph method [23] derives a new graph from the DAG and Z and runs the search as a declarative query or a user-defined function. None of these algorithms reuses work across queries, so each query takes $O(n + m)$ time on a DAG with n vertices and m edges.

However, real-world causal DAGs have grown to millions of variables [12, 55, 56], so a causal analysis that runs these algorithms for many d-separation queries takes time proportional to the size of the DAG times the number of queries. Since the DAG is fixed while the queries vary, such analyses call for an index that is built once and answers every query. As d-separation asks whether some path is open, a natural approach is to reduce it to reachability, for which graph indexes are well studied: 2-hop labels [1, 7, 50] and tree-decomposition indexes [18, 21] answer reachability without a traversal, and fault-tolerant reachability oracles [4, 6] even let a query delete a few vertices or edges, much as conditioning blocks the paths through a vertex. Adapting these indexes to d-separation, however, is non-trivial. To the best of our knowledge, no existing index supports d-separation with arbitrary conditioning sets. In this paper, we aim to fill this gap: we study how to index a causal DAG once so that every d-separation query is answered exactly, although X, Y, and Z are given only at query time.

► **Challenges.** Building such an index is difficult because of three properties of d-separation queries, which also prevent a direct reduction to reachability.

Ⓒ1 *Arbitrary and non-monotone conditioning.* The conditioning set Z is known only at query time and can be any set of vertices. Conditioning is moreover neither monotone nor local. In Figure 1, Drug and Gene are d-separated when nothing is observed, and adding Biomarker to Z makes them d-connected through a path that avoids Biomarker. An index built without knowing Z must therefore capture every path that some Z may open. This raises a first question: *Can an index built without knowing Z handle conditioning that opens paths as well as blocks them?*

Ⓒ2 *All-path separation.* Separation requires every path between X and Y to be blocked. A reachability label can show that a path exists in $\mathcal{G}$, but not whether some path remains open given Z, since the label does not record the vertices of the path. Deciding separation thus falls back on a traversal, which reports separation only after visiting every vertex that an open path from X reaches, possibly most of the DAG. This raises a second question: *Can we decide whether some path remains open without visiting all these vertices?*

Ⓒ3 *Query-dependent ignored edges.* Many causal analyses test d-separation while ignoring the edges into or out of some variables. In particular, the criteria that decide whether a causal effect can be estimated from observational data, such as the back-door criterion and the rules of the do-calculus [3, 24, 25, 28], rely on such tests. The edges to ignore depend on the sets of each query, so one DAG yields exponentially many modified graphs, and indexing each of them is infeasible. A single index built on $\mathcal{G}$ avoids this, but the paths that it records may use ignored edges, which are too many per query to exclude individually. This raises a third question: *Can an index built on $\mathcal{G}$ answer d-separation queries that ignore edges?*

► **Our technical contributions.** To answer these questions, we propose TrekIndex, the first index for d-separation with arbitrary conditioning sets and ignored edges, and hence for the criteria built on d-separation. Its key idea is to separate the two roles that a conditioned vertex plays on a path: as a non-collider, it blocks the path, and as a collider or a descendant of one, it opens the path. Blocking only removes paths, so TrekIndex handles it on a DAG that does not depend on Z; opening adds paths, so TrekIndex handles it with a small graph explored at query time.

To address Ⓒ1, we cut every open path at the conditioned vertices that open it. If a conditioned descendant of a collider opens the path, we first extend the path from the collider down to this descendant and back, which makes the descendant a collider. Each resulting piece, called a *trek*, has no collider, so conditioning can only block it. We encode all treks of $\mathcal{G}$ as the paths of a *trek DAG* (Figure 2). Since this DAG does not depend on Z, it is indexed once. Conditioning on a vertex then only forbids its states, which blocks every trek through it. At query time, a small *query graph* over $X \cup Y \cup Z$ joins treks at conditioned vertices and so restores the opened paths. Since each edge of the query graph is a *forbidden-set reachability* test on the trek DAG, every d-separation query reduces to such tests.

To address Ⓒ2, we decide a test from a state s to a state t by checking the forbidden states before searching the paths. A state lies between s and t if s reaches it and it reaches t. If s reaches t and no forbidden state lies between them, every path from s to t avoids the forbidden states, and the test succeeds. Both conditions are plain reachability questions, which 2-hop labels answer without a search. Bitmasks computed once per query check all forbidden states together. If a forbidden state lies between s and t, TrekIndex searches, but only among the states between them. To keep the labels small, *witness contraction* replaces the paths through the tree-like periphery of $\mathcal{G}$ by shortcut arcs that conditioning can still block. TrekIndex labels only the remaining core.

To address Ⓒ3, we turn ignored edges into forbidden states, as we do for conditioning. Ignoring the edges into or out of a vertex forbids two of its states, however many such edges it has, and restricts where a test may start and end. Some criteria also test for directed paths, e.g., whether Z contains a descendant of X. A directed path has no collider and is thus a trek, so these tests are also reachability tests on the trek DAG. Hence d-separation and the criteria built on it reduce to forbidden-set reachability on one DAG that no query modifies. A single index thus answers all of them.

In summary, we make the following contributions.

- We reduce d-separation with arbitrary conditioning sets and query-dependent ignored edges, and hence the criteria built on it, to forbidden-set reachability on a trek DAG built from $\mathcal{G}$ alone.
- We develop TrekIndex, to the best of our knowledge the first index for d-separation with arbitrary conditioning sets. It decides most tests by refuting or certifying them with labels, keeps its labels small by witness contraction, and serves the criteria built on d-separation with the same index; we prove that it answers every query exactly.

Table 1: Notations and meanings.

Notation	Meaning
$\mathcal{G} = (V, E)$, n, m	causal DAG, $n = \|V\|$, $m = \|E\|$
$\mathcal{G}_{\overline{A}\underline{C}}$	$\mathcal{G}$ with the edges into A and out of C ignored
Sep, Reach	d-separation test in $\mathcal{G}_{\overline{A}\underline{C}}$; directed-path test
$\mathcal{T}$; U_v, P_v, E_v, D_v	trek DAG; the four states of v
$\mathcal{S}(W)$; $s \rightsquigarrow_{\mathcal{F}} t$	states of W; path of $\mathcal{T}$ from s to t avoiding $\mathcal{F}$
$\mathcal{H}_Z$	query graph over $X \cup Y \cup Z$
K, w, $\mathcal{T}_w$	core, width, and witness DAG
$fwd(s)$, $bwd(t)$	forward and backward projections

- On six datasets with up to 12.2 million vertices, we compare TrekIndex with six traversal-based methods: the procedure of Geiger et al. [10], bidirectional and ancestral variants of Bayes-Ball [17, 35], RP-separation [5], CIfly [49], and the d-collision graph method [23]. Compared with the fastest of them, TrekIndex is (1) 4.84× to 6,935× faster on d-separation queries, (2) 69.6× to 5,552× faster on queries of eight criteria built on d-separation, and (3) 1.42× to 4,696× faster on three applications, namely covariate screening, literature replay, and the conditional-independence tests of the PC algorithm.

► **Outline.** Section 2 defines d-separation, the criteria built on it, and the problem, Section 3 reviews existing methods, Sections 4 to 6 present the trek DAG reduction, the index, and query processing, Section 7 reports the experiments, Section 8 reviews related work, and Section 9 concludes. Detailed proofs and additional results are deferred to the full version[1].

2 PRELIMINARIES

In this section, we define d-separation and the criteria built on it, and state the problem.

2.1 Causal DAGs, d-Separation, and Graph Surgery

A causal DAG $\mathcal{G} = (V, E)$ is a directed acyclic graph whose $n = |V|$ vertices are variables and which has $m = |E|$ edges. An edge $u \to v$ states that u is a direct cause, or *parent*, of v, and $\mathrm{Pa}(v)$ denotes the parents of v. A *path* is a sequence of distinct vertices in which consecutive vertices are adjacent, regardless of the directions of the edges. It is *directed* if all its edges point from its first vertex towards its last. We write $u \rightsquigarrow_F v$ if $u = v$ or $\mathcal{G}$ has a directed path from u to v whose inner vertices are not in a set F; its endpoints may be in F. If $u \rightsquigarrow v$, which abbreviates $u \rightsquigarrow_\emptyset v$, then u is an *ancestor* of v and v is a *descendant* of u. For a set S, $\mathrm{An}(S)$ denotes the ancestors of the vertices in S, including S itself. We write x for $\{x\}$ when a set has one vertex. Table 1 summarizes the frequently used notation.

An inner vertex v of a path is a *collider* if both edges of the path at v point into v, as in $u \to v \leftarrow w$, and a *non-collider* otherwise.

Definition 2.1 (d-separation [10, 26]). Let $X, Y, Z \subseteq V$ be pairwise disjoint, where Z is the *conditioning set*. A path is *blocked* by Z if (i) one of its non-colliders is in Z, or (ii) one of its colliders is not in $\mathrm{An}(Z)$, and *open* given Z otherwise. X and Y are *d-separated* by Z, written $(X \perp\!\!\!\perp Y \mid Z)$, if no path between a vertex of X and a vertex of Y is open given Z. Otherwise, X and Y are *d-connected* given Z.

A *d-separation query* asks whether $(X \perp\!\!\!\perp Y \mid Z)$ holds for sets X, Y, and Z given at query time.

Example 1. In Figure 1, every path between Drug and Gene has one of the colliders Protein, Recovery, and Disease. For Q_1, $Z = \{\text{Biomarker}\}$ contains no non-collider of these paths, and $\mathrm{An}(Z)$ contains Recovery but neither Protein nor Disease. Hence, the path Drug → Recovery ← Gene is open, and the answer to Q_1 is no.

Many criteria test d-separation after *graph surgery* [26], which ignores the edges into or out of some vertices.

Definition 2.2 (Graph surgery [26]). For $A, C \subseteq V$, $\mathcal{G}_{\overline{A}\underline{C}}$ is the graph with the vertices of $\mathcal{G}$ and every edge of $\mathcal{G}$ except those that enter a vertex of A or leave a vertex of C.

We omit A or C when it is empty: $\mathcal{G}_{\overline{A}}$ ignores only the edges into A, and $\mathcal{G}_{\underline{C}}$ only the edges out of C. We write $(X \perp\!\!\!\perp Y \mid Z)_{\overline{A}\underline{C}}$ for d-separation in $\mathcal{G}_{\overline{A}\underline{C}}$.

2.2 d-Separation-Based Queries

We call a criterion *d-separation-based* if it is decided by two kinds of *primitive tests*: $\mathrm{Sep}(X, Y, Z; A, C)$, which asks whether $(X \perp\!\!\!\perp Y \mid Z)_{\overline{A}\underline{C}}$ holds, and $\mathrm{Reach}(x, y; F)$, which asks whether $x \rightsquigarrow_F y$. Table 2 lists the nine such criteria that we study. As in prior work [14, 30], we state every criterion for sets of variables. Like a d-separation query, a *query* of a criterion applies it to input sets and holds if and only if all its tests in Table 2 hold; the sets are pairwise disjoint, except P of selection bias. We call the queries of these criteria *d-separation-based queries*.

The *back-door criterion* [26] guarantees that adjusting for Z identifies the effect of X on Y. It holds if Z contains no descendant of X and blocks every *back-door path* from X to Y, a path whose first edge points into X and whose other vertices are not in X [30]. Table 2 tests the blocking condition as $(X \perp\!\!\!\perp Y \mid Z)_{\underline{X}}$, which is equivalent when Z contains no descendant of X, as shown in the full version.

More generally, the three *rules of the do-calculus* [24] transform expressions such as $P(y \mid do(x), z, w)$, where $do(x)$ sets X to x. Rule 1 removes an observation z, rule 2 replaces an action $do(z)$ by the observation z, and rule 3 removes an action $do(z)$, each when its d-separation test in Table 2 holds. Rule 3 uses the vertices of Z that are not ancestors of W in $\mathcal{G}_{\overline{X}}$. These vertices are found by the tests $z \rightsquigarrow_X w$ in $\mathcal{G}$, since $\mathcal{G}_{\overline{X}}$ has no edge into X.

If a common cause of X and Y is unobserved, the effect can still be identified through intermediate variables M, such as Protein in Figure 1, by the *front-door criterion* [24]. When treatments are applied over time, the *sequential back-door criterion* [28] ensures that the joint effect of treatments $x_1, \ldots, x_k$ can be computed by adjusting for covariates W_i observed before each x_i. Here, $X_{<i}$ and $X_{>i}$ are the treatments before and after x_i, and $W_{\le i} = W_1 \cup \cdots \cup W_i$. When the question is how an effect arises, *mediation* analysis splits the effect of X on Y into a direct effect and an indirect effect through mediators M. Pearl's conditions [25, 27] ensure that both can be computed from data, and we use their graphical form, in which W

[1] https://github.com/HuiZ-W/TrekIndex/blob/main/paper/TrekIndex-full.pdf

Table 2: The nine criteria and the primitive tests that decide them.

Criterion	Input	d-separation tests (Sep)	Directed-path tests (Reach)
d-separation [26]	X, Y, Z	$(X \perp\!\!\!\perp Y \mid Z)$	–
Back-door [26]	X, Y, Z	$(X \perp\!\!\!\perp Y \mid Z)_{\underline{X}}$	no $x \rightsquigarrow z$ with $x \in X$, $z \in Z$
do-calculus rule 1 [24]	X, Y, Z, W	$(Y \perp\!\!\!\perp Z \mid X \cup W)_{\overline{X}}$	–
do-calculus rule 2	X, Y, Z, W	$(Y \perp\!\!\!\perp Z \mid X \cup W)_{\overline{X}\underline{Z}}$	–
do-calculus rule 3	X, Y, Z, W	$(Y \perp\!\!\!\perp Z \mid X \cup W)_{\overline{X}\,\overline{Z'}}$	Z': the $z \in Z$ with no $z \rightsquigarrow_X w$, $w \in W$
Front-door [14, 24]	X, Y, M	$(X \perp\!\!\!\perp M \mid \emptyset)_{\underline{X}}$; $(M \perp\!\!\!\perp Y \mid X)_{\underline{M}}$	no $x \rightsquigarrow_M y$ with $x \in X$, $y \in Y$
Sequential back-door [28]	$x_1, \ldots, x_k$; Y; $W_1, \ldots, W_k$	each i: $(Y \perp\!\!\!\perp x_i \mid X_{<i} \cup W_{\le i})_{\overline{X_{>i}}\,\underline{x_i}}$	each i: no $x \rightsquigarrow w$ with $x \in \{x_i\} \cup X_{>i}$, $w \in W_i$
Mediation [25, 27]	X, M, Y, W	$(M \perp\!\!\!\perp Y \mid X \cup W)_{\underline{M}}$; $(X \perp\!\!\!\perp M \mid W)_{\underline{X}}$; $(X \perp\!\!\!\perp Y \mid W)_{\overline{M}\underline{X}}$	no $v \rightsquigarrow w$ with $v \in X \cup M$, $w \in W$
Selection bias [3]	X, Y, P	$P \cap Y = \emptyset$ and $(Y \perp\!\!\!\perp P \setminus X \mid X)$	–

makes each term of the mediation formula computable by back-door adjustment.

Selection bias arises when the units in the data are not chosen at random. It is modeled by adding to $\mathcal{G}$ a selection vertex S without children, whose parents P affect selection. Then $P(y \mid x)$ is recoverable from the selected data if and only if $(Y \perp\!\!\!\perp S \mid X)$ holds in this graph [3]. Since P is given only at query time, S is not part of $\mathcal{G}$, and we test the condition on $\mathcal{G}$ as $P \cap Y = \emptyset$ and $(Y \perp\!\!\!\perp P \setminus X \mid X)$, which is equivalent, as shown in the full version.

2.3 Problem Definition

Since the primitive tests decide every criterion in Table 2, we define the problem over them.

Problem 1 (d-separation-based queries). *Given a DAG $\mathcal{G} = (V, E)$ in advance, answer the primitive tests* $\mathsf{Sep}(X, Y, Z; A, C)$ *and* $\mathsf{Reach}(x, y; F)$ *for arguments given only at query time, where X, Y, and Z are pairwise disjoint, A, C, $F \subseteq V$, and x, $y \in V$.*

Example 2. For Q_2 in Figure 1, $X = \{\text{Drug}\}$, $Y = \{\text{Disease}\}$, and $Z = \{\text{Age}\}$. By Table 2, Q_2 holds if and only if $\mathsf{Reach}(\text{Drug}, \text{Age}; \emptyset)$ is false and $\mathsf{Sep}(\text{Drug}, \text{Disease}, \text{Age}; \emptyset, \text{Drug})$ holds. They both hold: Age is not a descendant of Drug, and it blocks the only back-door path from Drug to Disease, Drug $\leftarrow$ Age $\rightarrow$ Disease.

As in prior work [14, 47], we assume that the causal DAG is given, e.g., by experts or causal discovery, and do not consider mixed graphs with latent confounders, updates to the DAG, or surgery on single edges, as in the proper back-door graph [30].

3 TRAVERSALS AND REACHABILITY INDEXES

In this section, we review traversal algorithms and reachability indexes and discuss their limitations for d-separation-based queries.

3.1 Online Traversal Algorithms

The linear-time algorithm of Geiger et al. [10] and Bayes-Ball [35] decide $(X \perp\!\!\!\perp Y \mid Z)$ by one search over pairs (v, d), where d records whether a path enters v from a parent or from a child. The search starts at X and passes a vertex $v \notin Z$ as a non-collider. Geiger et al. first mark $\mathrm{An}(Z)$ and turn at its vertices as colliders, whereas Bayes-Ball needs no marking: it turns only at conditioned vertices, and a path that descends from a collider to a conditioned descendant must turn upward again. In both, X and Y are d-connected if and only if the search reaches Y. Since every vertex of an open path between X and Y lies in $\mathrm{An}(X \cup Y \cup Z)$, other traversals [5, 17] search only this set. CIfly [49] encodes each criterion as a rule table over the same pairs and runs one generic search. In graph databases, the d-collision graph method [23] derives a mixed graph from $\mathcal{G}$ and Z and searches it. All of them evaluate $\mathsf{Sep}(X, Y, Z; A, C)$ by skipping ignored edges and $\mathsf{Reach}(x, y; F)$ by a forward search from x that skips F.

Complexity analysis. These online algorithms build no index, so no search reuses the work of another. On a DAG with n vertices and m edges, each primitive test takes $O(n + m)$ time and space in the worst case, and q tests take $O(q(n + m))$ time, which is prohibitive for a large number of tests on a large DAG.

3.2 Reachability Indexes

A reachability index precomputes labels or shortcuts on a fixed graph so that a query $u \rightsquigarrow v$ needs little or no search. Many labeling schemes build on 2-hop labeling [7], which assigns each vertex v an out-label $L_{out}(v)$ and an in-label $L_{in}(v)$, sets of *hubs* such that $u \rightsquigarrow v$ if and only if $L_{out}(u) \cap L_{in}(v) \neq \emptyset$. Pruned labeling [1, 16, 50] builds such labels by taking the vertices as hubs in a fixed order. From each hub, a forward and a backward search skip the vertices that the labels of earlier hubs already show to be reachable. Unlike 2-hop labels, which decide every query, interval labels such as GRAIL [51] refute most negative queries but fall back to a search otherwise. Tree-decomposition indexes [18, 21] eliminate vertices one at a time and connect the neighbors of each eliminated vertex by shortcuts that preserve reachability or distance. Other indexes accept path constraints in the query: a few forbidden vertices for fault-tolerant reachability oracles [4, 6], and a set of allowed edge labels for label-constrained reachability indexes [15, 29, 45].

None of these indexes records the intermediate vertices of the paths that it summarizes. Hence, a common hub of x and y does not show that some path from x to y avoids the conditioned vertices and the ignored edges. Nor can the constrained indexes forbid arbitrary vertices: label-constrained indexes would need one label per vertex, and their size grows with the number of labels. Moreover, indexes built on $\mathcal{G}$ do not capture the paths that conditioning opens: for Q_1, they report that Drug and Gene are mutually unreachable,

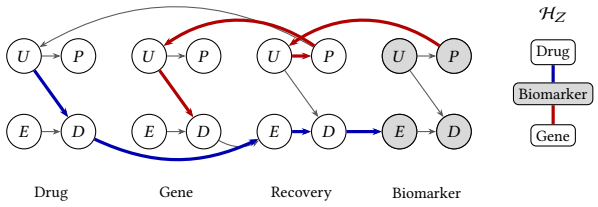


Figure 2: Part of the trek DAG of the running example and the query graph $\mathcal{H}_Z$ of Q_1.

although conditioning on Biomarker connects them. TrekIndex addresses both limitations. Its shortcut arcs keep the states they bypass, and per-query bitmasks over the hubs reveal the forbidden states between two states. A query graph explored at query time restores the paths that conditioning opens.

4 THE TREK DAG REDUCTION

In this section, we reduce every primitive test to forbidden-set reachability on a trek DAG built from $\mathcal{G}$ alone. The reduction has four steps.

4.1 Step 1: The Trek DAG

A *walk* is a sequence of vertices in which consecutive vertices are adjacent. Unlike a path, it may visit a vertex more than once, and each visit is a collider or a non-collider, defined as for paths. A walk is *d-connecting* if it has every collider in Z and every non-collider outside Z, as in Example 3. Two vertices x and y are d-connected given Z if and only if some d-connecting walk joins them [47]. A walk without colliders is a *trek*, a slight generalization of the usual pair of directed paths with a common top [40]. It first moves against the edges, turns at most once at a vertex whose two walk edges both leave it, and then moves along the edges.

Definition 4.1 (Trek DAG). The trek DAG $\mathcal{T}$ of $\mathcal{G}$ has four states for every vertex v. A trek enters v at U_v when it moves up and at E_v when it moves down, and leaves v at P_v toward a parent and at D_v toward a child. The arcs of $\mathcal{T}$ are $P_c \to U_p$ and $D_p \to E_c$ for every edge $p \to c$ of $\mathcal{G}$, and the *internal arcs* $U_v \to P_v$, $E_v \to D_v$, and $U_v \to D_v$ of every vertex v.

The trek DAG $\mathcal{T}$ has $4n$ states and $2m + 3n$ arcs. Arcs into U states follow the edges of $\mathcal{G}$ backwards, and arcs into E states follow them forwards. The only arcs from an up state (U or P) to a down state (E or D) are the arcs $U_v \to D_v$, and no arc leads back. Hence, $\mathcal{T}$ is acyclic, and a path from U_x to D_y moves up, takes exactly one arc $U_v \to D_v$, and then moves down. It visits the vertices of a trek between x and y, and every trek arises in this way.

For states s, t and a set $\mathcal{F}$ of states, we write $s \rightsquigarrow_{\mathcal{F}} t$ if $\mathcal{T}$ has a path from s to t whose inner states are not in $\mathcal{F}$. Deciding $s \rightsquigarrow_{\mathcal{F}} t$ is *forbidden-set reachability* on $\mathcal{T}$. Forbidding more states can only remove paths, so, unlike d-separation, forbidden-set reachability is monotone in $\mathcal{F}$.

4.2 Step 2: Conditioning

To handle the opening role of a conditioned vertex, we cut a d-connecting walk at its colliders. Each piece is a trek whose inner vertices are non-colliders and hence not in Z. Two consecutive pieces meet at a collider $z \in Z$: the first enters z from a parent through $D_p \to E_z$, and the next leaves z for a parent through $P_z \to U_{p'}$. A conditioned vertex z thus starts a trek at P_z and ends one at E_z. A vertex $v \in X \cup Y$ starts a trek at U_v and ends one at D_v, since the internal arcs out of U_v and into D_v cover the treks that leave or enter v in either direction. For a set W of vertices, we write $\mathcal{S}(W)$ for the states of the vertices in W.

Definition 4.2 (Query graph). The query graph $\mathcal{H}_Z$ has the nodes $X \cup Y \cup Z$, and an edge between two distinct nodes a and b if $start(a) \rightsquigarrow_{\mathcal{S}(Z)} end(b)$, where $start(v) = U_v$, $end(v) = D_v$ for $v \in X \cup Y$, and $start(z) = P_z$, $end(z) = E_z$ for $z \in Z$.

Reversing $\mathcal{T}$ swaps U with D and P with E, so $\mathcal{H}_Z$ is undirected.

Theorem 4.3 ([31, 47]). *X and Y are d-connected given Z if and only if $\mathcal{H}_Z$ contains a path from a vertex of X to a vertex of Y.*

Proof sketch. The treks obtained by cutting a d-connecting walk at its colliders, which lie in Z, form a path of $\mathcal{H}_Z$ from X to Y. Conversely, concatenating the treks of a shortest such path gives a d-connecting walk. The complete proof is in the full version. □

Theorem 4.3 separates the two roles of a conditioned vertex z on a walk. As a non-collider, z blocks the walk, since forbidding $\mathcal{S}(Z)$ removes every trek through z. As a collider, z opens the walk, since $\mathcal{H}_Z$ joins two treks at z. A conditioned descendant of a collider does the same, as Example 3 shows. Each edge of $\mathcal{H}_Z$ is thus a monotone forbidden-set reachability test, and only the connectivity of $\mathcal{H}_Z$ depends on Z in a non-monotone way.

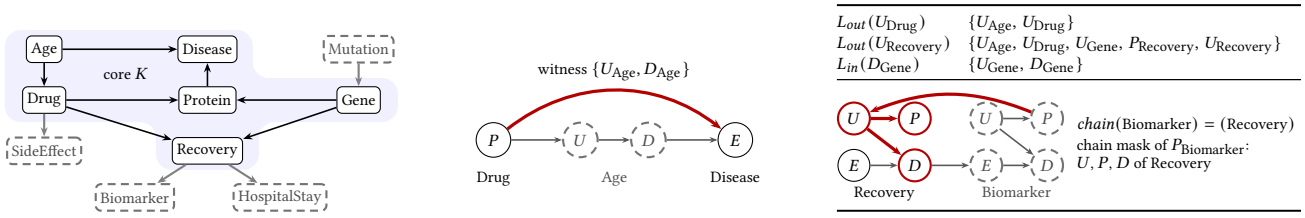
Example 3. In Figure 2, for Q_1 with $Z = \{\text{Biomarker}\}$, the states of Biomarker (shaded) may only start or end a trek. The blue trek Drug → Recovery → Biomarker gives the edge Drug–Biomarker of $\mathcal{H}_Z$, and the red trek Biomarker ← Recovery ← Gene gives the edge Biomarker–Gene. Together, they form the walk Drug → Recovery → Biomarker ← Recovery ← Gene, which goes down from Recovery, the collider of the path in Example 1, to its descendant Biomarker and back. On this walk, Recovery is a non-collider, and the only collider, Biomarker, is in Z, so Drug and Gene are d-connected, as found in Example 1.

4.3 Step 3: Ignored Edges

Since each edge $u \to v$ of $\mathcal{G}$ gives the arcs $P_v \to U_u$ and $D_u \to E_v$ of $\mathcal{T}$, ignoring the edges into a vertex $a \in A$ excludes the arcs out of P_a and into E_a, and ignoring the edges out of a vertex $c \in C$ excludes the arcs into U_c and out of D_c. Let $\mathcal{B}_{A,C} = \{P_a, E_a : a \in A\} \cup \{U_c, D_c : c \in C\}$. A path with an inner state in $\mathcal{B}_{A,C}$ uses one of these arcs, so surgery amounts to forbidding these states, except at the start and end of a path.

Lemma 4.4. *Let $\mathcal{T}_{\overline{A}\underline{C}}$ be the trek DAG of $\mathcal{G}_{\overline{A}\underline{C}}$. A path of $\mathcal{T}$ with at least one arc is a path of $\mathcal{T}_{\overline{A}\underline{C}}$ if and only if none of its inner states is in $\mathcal{B}_{A,C}$, it starts at neither P_a nor D_c, and it ends at neither E_a nor U_c, for any $a \in A$ and $c \in C$.*

Proof sketch. An inner state of a path has an arc in and an arc out, its start state only an arc out, and its end state only an arc in. The complete proof is in the full version. □



(a) Core and peeled vertices ($w = 1$). **(b) Witness shortcut arc of Age ($w = 2$).** **(c) Index entries of the running example ($w = 1$).**

Figure 3: Witness contraction and index of the running example.

By Theorem 4.3 applied to $\mathcal{G}_{\overline{AC}}$, $\text{Sep}(X, Y, Z; A, C)$ holds if and only if its query graph has no path from X to Y, and by Lemma 4.4, its edges are tested on the unmodified $\mathcal{T}$ with the forbidden set $\mathcal{S}(Z) \cup \mathcal{B}_{A,C}$. The start and end conditions never exclude U_v and D_v of $v \in X \cup Y$, and exclude P_z and E_z of $z \in Z$ exactly when $z \in A$; then z has no edge in $\mathcal{H}_Z$.

4.4 Step 4: Directed-Path Tests

The Reach tests of Table 2 ask for directed paths of $\mathcal{G}$, which are treks that never move up.

Lemma 4.5. *For vertices $x \neq y$ and a set F of vertices, $x \rightsquigarrow_F y$ in $\mathcal{G}$ if and only if $D_x \rightsquigarrow_{\mathcal{S}(F)} E_y$ in $\mathcal{T}$.*

Proof sketch. A path of $\mathcal{T}$ from D_x to E_y stays among E and D states and follows a directed path from x to y, whose inner vertices give its inner states. The complete proof is in the full version. □

By Lemma 4.5 and Step 3, every query of Table 2 thus reduces exactly to forbidden-set reachability tests on the same $\mathcal{T}$. A query never modifies $\mathcal{T}$. It only chooses where each test starts and ends and which inner states are forbidden: all four states of every vertex of Z and of F, P_v and E_v for $v \in A$, and U_v and D_v for $v \in C$.

5 INDEXING THE TREK DAG

In this section, we present the index structure and its construction.

5.1 Index Structure

Labeling all $4n$ states of $\mathcal{T}$ wastes space and time on large causal DAGs, in which many vertices lie in trees attached to the rest of the graph. TrekIndex therefore labels only the states of a core, which it obtains by *witness contraction*. Like the core–tree decomposition of CT-Index [18], witness contraction peels the tree-like periphery of the graph, but the shortcut arcs that replace paths across the periphery keep the states they bypass, since a conditioned vertex in the periphery must still block the paths through it.

Given a width $w \geq 1$, TrekIndex splits the vertices of $\mathcal{G}$ into a *core* K and a tree-like periphery of *peeled* vertices, eliminated one at a time, each when it has at most w neighbors in the *skeleton*, the underlying undirected graph of $\mathcal{G}$. The neighbors that a vertex v has when it is eliminated are its *portals*. Every path from v to the core passes a portal, and the *chain* $chain(v)$ of v collects its portals, their portals, and so on, up to the core.

Witness DAG. An arc between core states may bypass states of peeled vertices. Its *witness* is the set of these states. Given a set $\mathcal{F}$ of forbidden states, the arc is *open* if its witness avoids $\mathcal{F}$, and *blocked* otherwise. A path is *$\mathcal{F}$-free* if its arcs are open and its inner states avoid $\mathcal{F}$. A *self-detour* leaves a vertex c and returns to it, such as $P_{\text{Drug}} \to U_{\text{Age}} \to D_{\text{Age}} \to E_{\text{Drug}}$. Inside a path, it can be replaced by the internal arc $U_c \to D_c$, so only a path that starts at P_c or ends at E_c needs it, and the index drops the self-detours through peeled vertices. For $w = 1$, a peeled vertex has at most one portal, so a path between core states enters the periphery only in a self-detour, and no shortcut arc arises. A triple $(s, t, \mathcal{F})$, with s and t states of distinct vertices and $\mathcal{F}$ a set of states, is *admissible* if (i) $E_c \in \mathcal{F}$ or $D_c \in \mathcal{F}$ when $s = P_c$, and (ii) $U_c \in \mathcal{F}$ or $P_c \in \mathcal{F}$ when s is a U or P state and $t = E_c$; (i) blocks a path from P_c once its self-detour returns to c, and (ii) a path to E_c before its self-detour leaves c, so dropping self-detours loses no path. As the proof of Theorem 6.3 shows, every test $s \rightsquigarrow_{\mathcal{F}} t$ that a query runs is admissible.

Definition 5.1 (Witness DAG). A *witness DAG* $\mathcal{T}_w$ of $\mathcal{T}$ has the states of the core vertices and arcs with witnesses such that, for every admissible triple $(s, t, \mathcal{F})$ of core states, $s \rightsquigarrow_{\mathcal{F}} t$ in $\mathcal{T}$ if and only if $\mathcal{T}_w$ has an $\mathcal{F}$-free path from s to t.

Core labels. TrekIndex stores pruned 2-hop labels on the states of $\mathcal{T}_w$. For core states s and t, $L_{out}(s) \cap L_{in}(t) \neq \emptyset$ if and only if $s \rightsquigarrow t$ in $\mathcal{T}_w$.

Peripheral data. A peeled vertex has no labels. The *local arcs* of a peeled vertex v are the arcs, with witnesses, between the states of v and those of its portals at the moment v is eliminated. For every peeled vertex v, TrekIndex stores its local arcs, its chain, and the arcs of $\mathcal{T}_w$ whose witnesses contain states of v, so that a query that forbids such a state can block them at once. It also stores *chain masks* that, like the ancestor arrays of H2H [21], let a query follow a state of v to the core without a search: for each state s of v and each chain vertex u, two 4-bit masks hold the states of u that s reaches, and that reach s, by internal and local arcs.

Definition 5.2 (TrekIndex). The index of TrekIndex consists of (i) a witness DAG $\mathcal{T}_w$ of $\mathcal{T}$, (ii) the 2-hop labels of its states, and (iii) the peripheral data of the peeled vertices.

Example 4. Figure 3 shows the index of the running example for $w = 1$. Mutation, SideEffect, Biomarker, and HospitalStay are peeled, and $\mathcal{T}_w$ is $\mathcal{T}$ restricted to the other six vertices (Figure 3a). In Figure 3c, $L_{out}(U_{\text{Recovery}}) \cap L_{in}(D_{\text{Gene}}) = \{U_{\text{Gene}}\}$ reflects the trek Recovery ← Gene. The peripheral data of Biomarker are its chain (Recovery), its local arcs $P_{\text{Biomarker}} \to U_{\text{Recovery}}$ and $D_{\text{Recovery}} \to E_{\text{Biomarker}}$, and masks recording that $P_{\text{Biomarker}}$ reaches U_{Recovery}, P_{Recovery}, and D_{Recovery}.

Algorithm 1: Index construction

Input: a causal DAG $\mathcal{G} = (V, E)$
Output: the index of Definition 5.2

1 $w \leftarrow 3$ if width 1 eliminates at least $0.8|V|$ vertices, else $w \leftarrow 1$;
2 $\mathcal{T}' \leftarrow \mathcal{T}$;
3 **while** *some remaining vertex has skeleton degree at most* w **do**
4 $v \leftarrow$ a remaining vertex of minimum skeleton degree;
5 $\Pi(v) \leftarrow$ the portals of v, its current skeleton neighbors;
6 record the arcs between $\mathcal{S}(v)$ and $\mathcal{S}(\Pi(v))$ as local arcs of v;
7 **foreach** $e_1 = \alpha \to \sigma$ *into* $\mathcal{S}(v)$ *and* $e_2 = \sigma' \to \beta$ *out of* $\mathcal{S}(v)$ **do**
8 **if** $\sigma \rightsquigarrow \sigma'$ *inside* v *and* α, β *belong to different vertices* **then**
9 add $\alpha \to \beta$ to $\mathcal{T}'$ with witness $\mathcal{W}(e_1) \cup \mathcal{W}(e_2) \cup \{$states of v from σ to $\sigma'\}$;
10 remove v and $\mathcal{S}(v)$, and connect $\Pi(v)$ by fill edges;
11 $\mathcal{T}_w \leftarrow \mathcal{T}'$; build pruned 2-hop labels on $\mathcal{T}_w$;
12 **foreach** *peeled vertex* v **do** store $chain(v)$, its local arcs and chain masks, and the arcs whose witnesses meet $\mathcal{S}(v)$;
13 **return** $\mathcal{T}_w$, *its labels, and the peripheral data*;

5.2 Index Construction

Algorithm 1 shows the construction.

Peeling. TrekIndex repeatedly eliminates a vertex of minimum skeleton degree, as long as this degree is at most w (lines 3–5). We write v_i for the i-th eliminated vertex. When a vertex is eliminated, its portals are connected into a clique by *fill edges* (line 10), so the skeleton keeps every connection through it. The vertices that remain form the core. A larger width also eliminates vertices with up to w portals, which shrinks the core further but adds shortcut arcs at the portals that queries must scan. This helps only on tree-like graphs, so TrekIndex uses $w = 3$ if $w = 1$ eliminates at least 80% of the vertices, and $w = 1$ otherwise (line 1).

Witness contraction. TrekIndex contracts $\mathcal{T}$ as the vertices are eliminated. Let $\mathcal{T}^{(0)} = \mathcal{T}$, and write $\mathcal{W}(e)$ for the witness of an arc e, which is empty for the arcs of $\mathcal{T}$. When v_i is eliminated, $\mathcal{T}^{(i)}$ is obtained from $\mathcal{T}^{(i-1)}$ by replacing the states of v_i with *shortcut arcs*. Consider an arc $e_1 = (\alpha \to \sigma)$ into a state of v_i and an arc $e_2 = (\sigma' \to \beta)$ out of one, where σ reaches σ' by internal arcs of v_i. If α and β belong to different vertices, $\mathcal{T}^{(i)}$ receives the shortcut arc $\alpha \to \beta$ (lines 7–9), whose witness is $\mathcal{W}(e_1) \cup \mathcal{W}(e_2)$ together with the states of v_i from σ to σ'. Otherwise, the pair forms a self-detour and is dropped. Different pairs may give parallel arcs with the same ends but different witnesses. TrekIndex keeps all of them, since a query may block one but not another. By the following lemma, the result $\mathcal{T}_w$ is a witness DAG.

Lemma 5.3. *Let $(s, t, \mathcal{F})$ be an admissible triple whose ends s and t belong to vertices other than $v_1, \ldots, v_i$. Then $s \rightsquigarrow_{\mathcal{F}} t$ in $\mathcal{T}$ if and only if $\mathcal{T}^{(i)}$ has an $\mathcal{F}$-free path from s to t.*

Proof sketch. By induction on i. Expanding the shortcut arcs added for v_i turns an $\mathcal{F}$-free path of $\mathcal{T}^{(i)}$ into one of $\mathcal{T}^{(i-1)}$. A shortest $\mathcal{F}$-free path of $\mathcal{T}^{(i-1)}$ visits v_i at most once and, by admissibility, not as a self-detour; $\mathcal{T}^{(i)}$ replaces this visit by an open shortcut arc. The complete proof is in the full version. □

Example 5. With $w = 2$, Age is also eliminated, with portals Drug and Disease. Its arcs $P_{\text{Drug}} \to U_{\text{Age}} \to D_{\text{Age}} \to E_{\text{Disease}}$ compose the shortcut arc $P_{\text{Drug}} \to E_{\text{Disease}}$ with witness $\{U_{\text{Age}}, D_{\text{Age}}\}$ (Figure 3b), which stands for the trek Drug ← Age → Disease. In Q_2, Age is conditioned, so the witness blocks the arc, which would otherwise wrongly connect Drug and Disease.

Labels and peripheral data. As in distribution labeling [16], the labeling of $\mathcal{T}_w$ (line 11) processes hubs in decreasing order of $(d^- + 1)(d^+ + 1)$, where d^- and d^+ are the in-degree and the out-degree of a state in $\mathcal{T}$ restricted to the core. The chain masks come from one search per state with nothing forbidden (line 12).

Complexity. Let ℓ be the maximum label size, m_w and m_s the numbers of arcs of $\mathcal{T}_w$ and of shortcut arcs created during elimination, and f the number of fill edges. Elimination takes $O((n + m + f)\log n)$ time plus the examination of the arc pairs at each eliminated vertex, and pruned labeling $O(\ell^2 m_w)$ time. The index takes $O(|K|\ell + m_w + m_s)$ space, and $O(\sum_v(|chain(v)| + l_v + c_v))$ for the peripheral data, where l_v and c_v count the local arcs of v and the arcs of $\mathcal{T}_w$ that v witnesses.

6 QUERY PROCESSING

In this section, we show how TrekIndex answers the queries of Table 2. A query runs reachability tests on $\mathcal{T}$, exploring a query graph for each Sep test (Section 6.1). Each reachability test maps its two ends onto the core (Section 6.2) and then decides pairs of core states (Section 6.3). Section 6.4 proves that the answers are exact.

6.1 Exploring the Query Graph

We call each Sep test of Table 2, and each set of Reach tests in its last column, a *condition* of a query. A condition runs forbidden-set reachability tests $s \rightsquigarrow_{\mathcal{F}} t$ that share one forbidden set $\mathcal{F}$; we call them *edge tests* after the function EdgeTest that decides them. A query first evaluates its Reach conditions and then its Sep conditions in the order of Table 2, and stops at the first condition that fails (lines 2–5 of Algorithm 2); the sequential back-door criterion runs these lines for each treatment in turn. Rule 3 computes Z' before line 2 by testing, for every $z \in Z$, whether $z \rightsquigarrow_X u$ for some $u \in W$. A Sep condition asks whether $\mathcal{H}_Z$ connects X and Y. Since each edge of $\mathcal{H}_Z$ is an edge test, TrekIndex explores $\mathcal{H}_Z$ from one side and tests only the edges at the nodes it reaches, without building $\mathcal{H}_Z$ in full. Function SepTest of Algorithm 2 shows this exploration. It first marks the forbidden states of Section 4.4 and blocks the shortcut arcs whose witnesses contain them (line 8). It then explores $\mathcal{H}_Z$ depth-first from a side S, testing the edges to the other side first, and stops when it reaches that side (lines 9–16).

Live states. If $v \in X \cup Y$ is in $C \setminus A$, its start and end states U_v and D_v are forbidden as inner states, so a path from U_v must take $U_v \to P_v$ and a path into D_v must come from E_v. TrekIndex therefore starts the edge tests of v at P_v and ends them at E_v, the *live* states of v, without losing any path. Symmetrically, if $v \in A \setminus C$, the edge tests of v start at D_v and end at U_v. A vertex of $A \cap C$ or of $A \cap Z$ has no live states and no edge; for the other vertices, the live states are the start and end states of Definition 4.2.

Surgery-first. Since $\mathcal{H}_Z$ is undirected, the exploration may start from either set of the Sep test: from the first, or from the second if $A \cup C$ meets it but not the first, as for rules 2 and 3 and the sequential

Algorithm 2: Answering queries of all criteria in Table 2

```
1  Function Query(q):
2      foreach pair (x, y) of a Reach condition "no x ⇝_F y" of q do
3          if EdgeTest(D_x, E_y) avoiding S(F) then return false;
4      foreach Sep condition (X, Y, Z; A, C), in the order of Table 2 do
5          if not SepTest(X, Y, Z; A, C) then return false;
6      return true;
7  Function SepTest(X, Y, Z; A, C):
8      mark the forbidden states and block the arcs they witness;
9      S ← Y if only Y meets A ∪ C, else X;    O ← the other set;
10     R ← S; push the vertices of S that have live states on a stack;
11     while the stack is not empty do
12         i ← pop the stack;
13         foreach j ∈ (X ∪ Y ∪ Z) \ R with live states, O first do
14             if EdgeTest(live start of i, live end of j) then
15                 if j ∈ O then return false;
16                 R ← R ∪ {j}; push j;
17     return true;
18 Function EdgeTest(s, t):
19     compute fwd(s) and bwd(t), each at most once per condition;
20     if fwd(s) ∩ bwd(t) has an allowed state then return true;
21     foreach dominant σ ∈ fwd(s), τ ∈ bwd(t) of distinct vertices
         do
22         if L_out(σ) ∩ L_in(τ) = ∅ then continue;
23         if the certificate holds then return true;
24         if the label-pruned search succeeds then return true;
25     return false;
```

back-door criterion (line 9). A vertex v of the second set in C or A starts its treks at P_v or D_v, respectively. They leave v only through its parents or only through its children, so the exploration may reach fewer nodes of $\mathcal{H}_Z$ from this side.

6.2 Decomposing a Test

Since a peeled vertex has no labels, EdgeTest(s, t) first computes the *projections* of its ends (line 19). For a core state s, the *forward projection* is $fwd(s) = \{s\}$. For a state s of a peeled vertex, $fwd(s)$ collects the states through which paths from s reach the core: s and every state that s reaches without passing a forbidden state, following the internal arcs of each reached vertex and the open local arcs of each reached peeled vertex. A forbidden state other than s is not expanded but is kept, since it may be the end t. All states of $fwd(s)$ belong to the vertex of s and its chain. The *backward projection* $bwd(t)$ is symmetric. The chain masks give the projection of a state of a peeled vertex v without a search whenever no state of v or of its chain is forbidden and the local arcs along the chain have empty witnesses. We call a state *allowed* if it is s or t or is not in $\mathcal{F}$. A path from s to t either passes a state of both projections or leaves $fwd(s)$ and enters $bwd(t)$ at core states, which gives Lemma 6.1.

Lemma 6.1. *Let $(s, t, \mathcal{F})$ be an admissible triple. Then $s \rightsquigarrow_{\mathcal{F}} t$ in $\mathcal{T}$ if and only if (i) $fwd(s) \cap bwd(t)$ contains an allowed state, or (ii) for some allowed core state $\sigma \in fwd(s)$ and some allowed core state $\tau \in bwd(t)$ of another vertex, $\mathcal{T}_w$ has an $\mathcal{F}$-free path from σ to τ.*

Proof sketch. Expanding the local and shortcut arcs gives a path of $\mathcal{T}$ whose inner states are allowed or lie in open witnesses. Conversely, a shortest path gives (i), or its first and last core states form an admissible triple with $\mathcal{F}$ and give (ii) by Lemma 5.3. The complete proof is in the full version. □

EdgeTest checks (i) first (line 20). For (ii), it tries only the *dominant* states (line 21), allowed core states of $fwd(s)$ that no allowed state of the same vertex in $fwd(s)$ dominates: U_c dominates P_c and D_c, and E_c dominates D_c, since an internal arc leads from the dominating state to the dominated one. Each core vertex thus has at most two dominant states in $fwd(s)$, and symmetrically in $bwd(t)$.

6.3 Testing Core States

EdgeTest decides (ii) for two core states σ and τ in three layers of increasing cost (lines 22–24). Every path of $\mathcal{T}_w$ that is not $\mathcal{F}$-free passes an *entry state*, which is of three kinds: U_z or E_z, through which paths enter a core vertex z whose states are all forbidden; a state of $\mathcal{B}_{A,C}$; or either end of a blocked shortcut arc. We write $\mathcal{E}$ for the set of entry states.

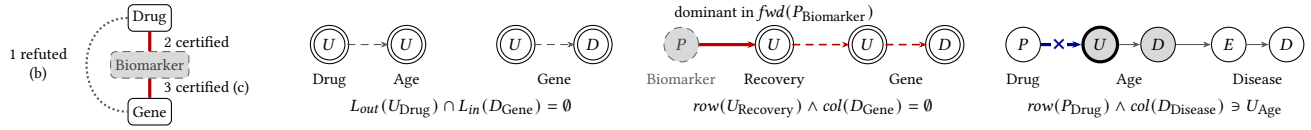
Refutation by labels. If $L_{out}(\sigma) \cap L_{in}(\tau) = \emptyset$, $\mathcal{T}_w$ has no path from σ to τ even with nothing forbidden. Since forbidden states and blocked arcs only remove paths, this pair fails.

Certification by hub masks. Suppose that paths exist, none of them passes a state of $\mathcal{E}$ other than σ and τ, and no blocked arc leads directly from σ to τ. Then every path is $\mathcal{F}$-free, so the edge test succeeds without examining any path. To check this, TrekIndex assigns every hub r two bitmasks over $\mathcal{E}$, its *hub masks*, once per condition. The mask $in(r)$ holds the states $\mu \in \mathcal{E}$ with $r \in L_{in}(\mu)$, which r reaches, and $out(r)$ those with $r \in L_{out}(\mu)$, which reach r. The union of $in(r)$ over $r \in L_{out}(\sigma)$ gives $row(\sigma)$, the states of $\mathcal{E}$ that σ reaches, and the union of $out(r)$ over $r \in L_{in}(\tau)$ gives $col(\tau)$, those that reach τ. Thus $row(\sigma) \wedge col(\tau)$ holds the entry states that lie between σ and τ. The certificate holds if no blocked arc leads directly from σ to τ and this set has none besides σ and τ. Computing a row or a column takes $O(\ell \cdot \omega)$ time, where $\omega = \lceil |\mathcal{E}|/64 \rceil$ is the number of words of a bitmask.

Label-pruned search. Otherwise, some state of $\mathcal{E}$ lies between σ and τ, and the labels cannot decide whether every path between them passes such a state. TrekIndex then runs a bidirectional search on $\mathcal{T}_w$ that skips forbidden states and blocked arcs, always expands the side whose frontier has fewer arcs, and keeps a forward state only if it still reaches τ and a backward state only if σ still reaches it. Each kept state u carries the entry states that still lie between u and the other end: $row(u) \wedge col(\tau)$ without τ for a forward state, and $row(\sigma) \wedge col(u)$ without σ for a backward state. This set contains u itself if u is an entry state. The search succeeds when the frontiers meet, or when this set becomes empty for a kept state u. Then u is not an entry state, so the certificate holds from u to the other end. It fails when a frontier becomes empty.

Lemma 6.2. *Let σ and τ be core states that EdgeTest tests. (a) If $L_{out}(\sigma) \cap L_{in}(\tau) = \emptyset$, $\mathcal{T}_w$ has no $\mathcal{F}$-free path from σ to τ. (b) If $L_{out}(\sigma) \cap L_{in}(\tau) \neq \emptyset$ and the certificate holds, such a path exists. (c) The label-pruned search succeeds if and only if such a path exists.*

Proof sketch. (a) holds by monotonicity, and (b) and the early stops of (c) hold by the property of $\mathcal{E}$. The search skips only states



(a) Exploration of $\mathcal{H}_Z$ in Q_1. **(b) Refutation in Q_1.** **(c) Projection and certificate in Q_1.** **(d) Search in Q_2.**

Figure 4: Exploration and edge tests of Q_1 and Q_2 for $w = 1$.

and arcs that no $\mathcal{F}$-free path from σ to τ uses. The complete proof is in the full version. □

Example 6. In Figure 4, gray states are forbidden, thick states are entry states, double circles are hubs, dashed arcs join states to their hubs, dotted lines are failed edge tests, red lines are certified edge tests and their paths, and crossed arcs lead to forbidden states. In Q_1, the exploration starts from Drug (Figure 4a). The edge test for Gene is refuted, since the labels of U_{Drug} and D_{Gene} share no hub (Figure 4b). The edge test for Biomarker is certified. From Biomarker, the core part of the edge test for Gene starts at the only dominant state of $fwd(P_{\text{Biomarker}})$, U_{Recovery}, which shares the hub U_{Gene} with D_{Gene}. Since Biomarker is peeled and $\mathcal{T}_w$ has no shortcut arc, $\mathcal{E}$ is empty, and the certificate holds (Figure 4c), so Drug and Gene are d-connected. In Q_2, the edges out of Drug are ignored, so U_{Drug} and D_{Drug} are forbidden, and the edge tests of Drug start at P_{Drug}, whose only arc leads to U_{Age}. The entry states are U_{Drug} and D_{Drug}, and U_{Age} and E_{Age}, through which paths enter the conditioned Age. The hub U_{Age} has the hub masks $in(U_{\text{Age}}) = \{U_{\text{Age}}, D_{\text{Drug}}\}$, the entry states that it reaches, and $out(U_{\text{Age}}) = \{U_{\text{Age}}, U_{\text{Drug}}\}$, those that reach it. For Disease, U_{Age} is a hub of both $L_{out}(P_{\text{Drug}})$ and $L_{in}(D_{\text{Disease}})$, so $row(P_{\text{Drug}})$ includes $in(U_{\text{Age}})$, and $col(D_{\text{Disease}})$ includes $out(U_{\text{Age}})$. Both contain U_{Age}, which thus lies between P_{Drug} and D_{Disease}, so the certificate fails, and so does the search (Figure 4d). The edge test for Age is refuted, so the condition holds.

Walled-in nodes. If Z contains most parents of a core node x, e.g., $Z = \text{Pa}(x)$, the entry states U_p of these parents make many certificates from x fail. For such a *walled-in* node, TrekIndex computes its *up-closure* and *turning* states once: the core U and P states that its start state reaches by $\mathcal{F}$-free paths, and the D and E states where these paths turn down. An edge test from x to t with no state of x in $bwd(t)$ then succeeds if an allowed state of $bwd(t)$ is in the closure or is a turning state, and otherwise runs its core tests from the turning states not in $\mathcal{F}$. Size thresholds are given in the full version.

6.4 Correctness and Complexity

Theorem 6.3. *(1) For every edge test from s to t that TrekIndex runs, EdgeTest returns true if and only if $s \rightsquigarrow_{\mathcal{F}} t$ in $\mathcal{T}$. (2) For every condition* $\text{Sep}(X, Y, Z; A, C)$*, SepTest returns true if and only if X and Y are d-separated by Z in $\mathcal{G}_{\overline{A}\underline{C}}$. Hence TrekIndex answers every query of Table 2 exactly.*

Proof sketch. (1) Every edge test is admissible: a Sep test starts at P_c or ends at E_c only if U_c and D_c are forbidden, and a Reach test starts at a D state. Dominant and turning states lose no path, so Lemmas 6.1 and 6.2 give (1). (2) Since live states lose no path, (1) shows that SepTest searches the query graph of $\mathcal{G}_{\overline{A}\underline{C}}$, so Theorem 4.3 and Lemma 4.4 give (2). The complete proof is in the full version. □

Table 3: Statistics of the datasets used in our experiments.

Graph	Type	Domain	$\lvert V\rvert$	$\lvert E\rvert$
BA 500k	Synthetic	Random graph	500,000	1,499,991
ASER	Causal	Event	1,310,917	8,657,788
iKraph	Causal	Biomedical	1,944,437	5,319,797
BA 2M	Synthetic	Random graph	2,000,000	5,999,991
CNFDAG	Causal	Web claim	5,394,394	7,423,212
CauseNet	Causal	Web claim	12,186,310	10,840,656

Complexity. Let $h = |X \cup Y \cup Z|$, κ the maximum chain length, and δ the maximum number of shortcut arcs at a vertex. Marking (line 8) takes time linear in the marked vertices and the shortcut arcs they witness, a refutation $O(\ell)$ time, and the hub masks $O(|\mathcal{E}| \cdot \ell \cdot \omega)$ time per condition. Given a row and a column, a certificate takes $O(\omega + \delta)$ time, and a search spends $O(\ell \cdot \omega)$ time per visited state and visits each state and arc of $\mathcal{T}_w$ at most once. A Sep condition tests at most h^2 edges and computes one row and one column per node. Hence, after marking, a Sep condition whose nodes are all core vertices and whose edge tests do not search takes $O(|\mathcal{E}| \cdot \ell \cdot \omega + h \cdot \ell \cdot \omega + h^2(\ell + \omega + \delta))$ time. A peeled node adds $O(\kappa(w + \delta))$ time for its projections, and each of its edge tests tries at most $4\kappa^2$ pairs of dominant states.

7 EXPERIMENTS

In this section, we present the experimental results. Section 7.1 describes the setup, Section 7.2 compares TrekIndex with the traversals, Section 7.3 analyzes its index and design choices, and Section 7.4 evaluates three applications.

7.1 Setup

► **Datasets.** We use six DAGs (Table 3). Four are real causal graphs extracted from text: ASER [55], the concept graph of an eventuality knowledge graph; iKraph [56], the directed relations of a biomedical knowledge graph; and CNFDAG [23] and CauseNet, two DAGs that break the cycles of the CauseNet knowledge base [12] differently. CNFDAG keeps only the largest weakly connected component. We keep an acyclic subgraph of every cyclic graph without reversing any edge. BA 500k and BA 2M are synthetic Barabási–Albert graphs [2] with three edges per new vertex, each pointing from the earlier to the later vertex of a random order.

► **Queries.** A query picks x uniformly among the non-isolated vertices and y as the end of a random walk of 1 to 6 steps from x in the skeleton, so that x and y are close. Its sets of k vertices are drawn uniformly (*random*) or as the ends of upward walks of

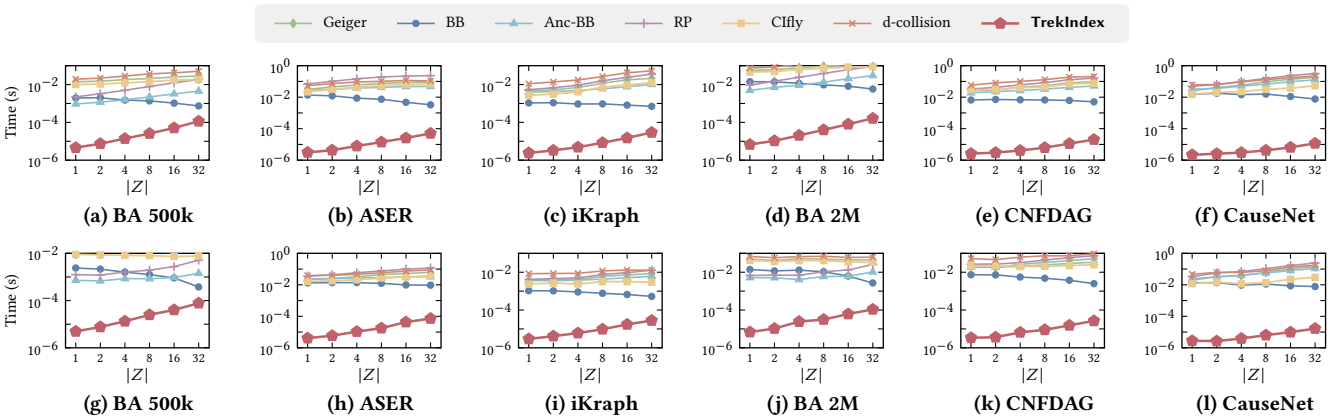


Figure 5: Mean d-separation query time for random Z ((a)–(f)) and ancestor Z ((g)–(l)).

1 to 8 steps from x or y (*ancestor*). d-Separation uses $|Z| = k \in \{1, 2, 4, 8, 16, 32\}$. For the criteria, $X = \{x\}$, $Y = \{y\}$, and one set has $k \in \{1, 4, 16\}$ vertices: Z for the back-door criterion, M for the front-door criterion, P for selection bias, and W for the rules and mediation, whose Z and M have one vertex. The sequential back-door criterion has the treatments x and one more vertex and splits k covariates between W_1 and W_2. Each run answers 1,000 queries of one configuration after a warm-up with 1,000 different queries, and we report the mean time per query, which includes all the work of a query, such as marking forbidden states and building masks, but not index construction.

► **Competitors.** We compare TrekIndex with six traversals.

(1) *Geiger* [10]: the procedure of Geiger et al., which marks $\mathrm{An}(Z)$ and then searches from one end.
(2) *Bidirectional Bayes-Ball* (BB) [35]: Bayes-Ball, which needs no marking, searched from both ends, expanding the cheaper side.
(3) *Ancestral Bayes-Ball* (Anc-BB) [17]: BB restricted to $\mathrm{An}(X \cup Y \cup Z)$, computed per query.
(4) *RP-separation* (RP) [5]: a search from one end that visits only the ancestors of $X \cup Y \cup Z$.
(5) *CIfly* [49]: the rules of Bayes-Ball as a CIfly rule table, searched from one end as in Bayes-Ball; we port CIfly to C++.
(6) *d-collision* [23]: a graph-database method that marks $\mathrm{An}(Z)$ and searches a mixed graph derived on the fly from $\mathcal{G}$ and Z, written as openCypher queries or a Neo4j user-defined function; we port the latter to C++.
(7) *TrekIndex*: our index, built once per graph.

The traversals stop at the first path between the two ends of a test, reuse their workspace across queries, skip the ignored edges, and evaluate the conditions of Table 2 in the same order as TrekIndex.

► **Settings.** All methods are implemented in C++, compiled with GCC 13.4 (`-O3`), and run single-threaded on a server with two Intel Xeon Gold 6226R CPUs (2.9 GHz) and 512 GB of memory. All methods answer the same queries, and every answer agrees with that of BB.

7.2 Comparison with Existing Methods

► **Exp-1: d-Separation.** Figure 5 reports the mean query time of all methods. TrekIndex is faster than the fastest baseline in all 72 configurations, by 4.84× (BA 500k, ancestor, $|Z| = 32$) to 6,935× (CauseNet), and by at least 19.0× on the graphs with millions of vertices. The speedup shrinks as $|Z|$ grows. A larger Z enlarges $\mathcal{H}_Z$, so a query performs more edge tests and more searches, whereas BB, the fastest baseline in most configurations, stops earlier when more vertices block it. Across graphs, the speedup is smallest on BA 500k, which has no periphery to peel. It is largest on the graphs whose periphery holds more than 95% of the vertices (Table 4), at least 93.3× on CNFDAG and 466× on CauseNet, since the projections decide many tests there without reaching the core.

Table 4: Index construction of TrekIndex on one thread.

Graph	Width w	Peeled vertices	Core vertices	Build time	Label entries	Index size
BA 500k	1	0.0 %	500 k	46.7 s	116 M	481 MiB
ASER	1	37.9 %	814 k	62.0 s	139 M	606 MiB
iKraph	1	59.2 %	793 k	43.7 s	106 M	496 MiB
BA 2M	1	0.0 %	2.00 M	371 s	681 M	2.69 GiB
CNFDAG	3	95.7 %	230 k	21.8 s	21.4 M	286 MiB
CauseNet	3	98.2 %	218 k	75.4 s	99.7 M	736 MiB

► **Exp-2: Causal criteria.** Figure 6 reports the mean query time of every method over random sets of 1, 4, and 16 vertices. TrekIndex is faster than the fastest baseline for all 48 combinations of a criterion and a graph, by 69.6× (back-door, BA 500k) to 5,552× (rule 3, CauseNet). As in Exp-1, the speedup depends mainly on the graph: for every criterion, it is smallest on BA 500k, which has no periphery, and at least 515× on the graphs whose periphery holds more than 95% of the vertices.

7.3 Detailed Analysis of TrekIndex

► **Exp-3: Index construction.** Table 4 reports the index construction. Build times are means of three builds. TrekIndex is built on one thread within 6.2 minutes on every dataset. The slowest is BA 2M, which has no periphery, so all its states are labeled. The index size counts the labels and the peripheral data of Section 5.1, but not the input graph and its core subgraph, on which the searches run. It ranges from 286 MiB to 2.69 GiB. Compared with 2-hop labels on

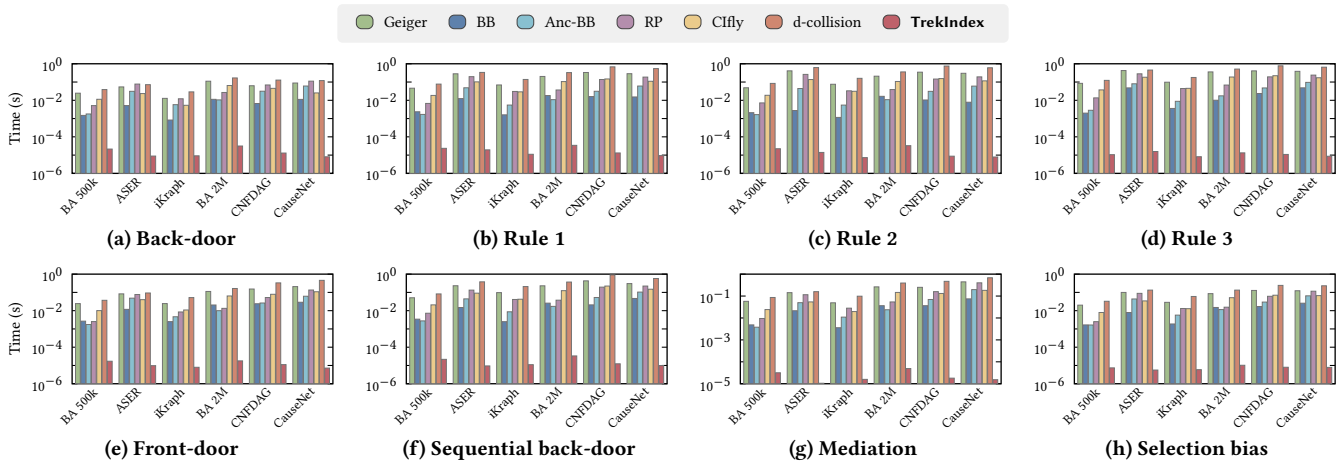


Figure 6: Mean query time of the causal criteria for random sets of 1, 4, and 16 vertices.

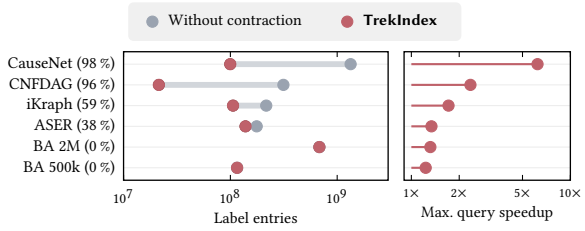


Figure 7: Effect of witness contraction.

Table 5: Mean query time with and without the index.

Graph	Z	$\lvert Z\rvert = 1$ BB ms	Trav. ms	Search ms	TrekIndex µs	$\lvert Z\rvert = 32$ BB ms	Trav. ms	Search ms	TrekIndex µs
BA 500k	Rand.	1.23	1.80	3.17	**4.54**	0.439	0.721	63.8	**85.6**
	Anc.	1.44	2.27	2.49	**4.78**	0.231	0.360	12.6	**66.2**
ASER	Rand.	10.5	13.7	17.2	**2.97**	2.19	4.31	95.7	**44.5**
	Anc.	10.3	12.7	13.7	**3.97**	7.26	9.55	79.5	**63.9**
iKraph	Rand.	0.821	0.879	1.16	**2.38**	0.553	0.578	17.7	**27.2**
	Anc.	0.807	0.918	0.975	**2.91**	0.412	0.339	4.08	**25.4**
BA 2M	Rand.	10.5	13.2	21.8	**6.15**	3.96	4.90	440	**132**
	Anc.	10.5	13.1	13.9	**6.40**	1.75	2.00	81.6	**94.0**
CNFDAG	Rand.	6.10	5.86	8.43	**2.60**	4.41	3.93	131	**19.2**
	Anc.	6.59	6.17	7.07	**3.33**	2.15	2.04	42.8	**24.3**
CauseNet	Rand.	13.7	14.1	15.4	**2.25**	6.69	7.37	90.2	**11.0**
	Anc.	12.0	12.2	13.2	**2.80**	6.77	7.39	65.0	**15.6**

the full trek DAG, witness contraction makes the construction up to 7.48× faster and the index up to 8.18× smaller (the full version).

► **Exp-4: Effect of the index.** To separate the index from the reduction, we run two variants without an index. *Trek traversal* runs one bidirectional search per query in $\mathcal{T}$, in which the states of conditioned vertices are forbidden except that a path entering $z \in Z$ from a parent may leave it upward. *Trek search* explores $\mathcal{H}_Z$ as TrekIndex does and answers each edge test by a bidirectional search in $\mathcal{T}$. Table 5 compares them with BB and TrekIndex on the d-separation workloads of Exp-1 with $|Z| = 1$ and 32, all measured in the same process. Trek traversal, which explores the same paths as BB, takes 0.824 to 1.97 times as long as BB. Trek search is slower than BB in every configuration, by 1.07× to 145×. The gap grows with $|Z|$: each edge test is a search, and a larger Z enlarges $\mathcal{H}_Z$, so a query performs more edge tests. TrekIndex answers the same queries 5.44× to 6,280× faster than trek traversal and 161× to 8,175× faster than trek search. The speedups of TrekIndex thus come from the index, not from the reduction alone.

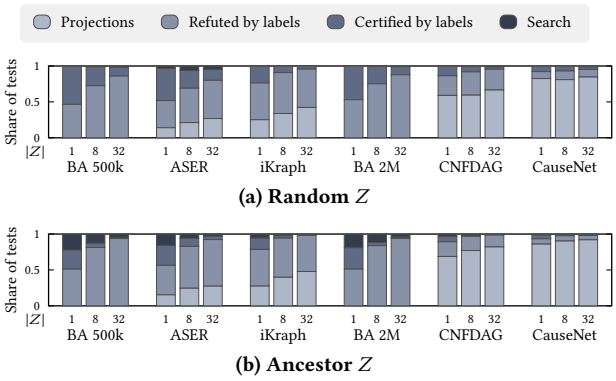
Table 6: Speedup from surgery-first exploration on rule 2.

Graph	Random $k=1$	Random $k=4$	Random $k=16$	Ancestor $k=1$	Ancestor $k=4$	Ancestor $k=16$
BA 500k	1.05	1.13	1.18	1.40	2.46	3.36
ASER	1.11	1.79	2.64	1.82	6.36	7.43
iKraph	1.09	1.43	1.81	1.42	2.38	2.70
BA 2M	1.01	1.15	1.18	1.37	2.49	4.37
CNFDAG	1.08	1.48	1.80	1.27	1.53	2.28
CauseNet	1.04	1.11	1.28	1.12	1.41	1.78

► **Exp-5: Effect of witness contraction.** We compare TrekIndex with a variant that skips witness contraction and builds 2-hop labels on the full trek DAG in Figure 7. On graphs with a large periphery, witness contraction reduces the number of label entries from 314 million to 21.4 million on CNFDAG (14.7×) and from 1.34 billion to 99.7 million on CauseNet (13.4×). Queries become faster as well, by up to 2.35× on CNFDAG and 6.22× on CauseNet, since the projections decide many tests without labels.

► **Exp-6: Effect of surgery-first exploration.** Table 6 reports the speedup of surgery-first on rule 2, the ratio of the mean query times of TrekIndex without and with it; the full version adds rule 3. On rule 2, surgery-first speeds up every configuration, by up to 7.43×. The gain grows with k, and it is larger for ancestor sets than for random sets of the same size.

► **Exp-7: Where tests are decided.** Figure 8 splits the edge tests of d-separation queries by how TrekIndex decides them: by the projections alone, by labels that refute or certify a test, or by a



(a) Random Z

(b) Ancestor Z

Figure 8: How TrekIndex decides d-separation edge tests.

Table 7: Total query time of the application workloads.

Graph	Queries	Geiger	BB	Anc-BB	RP	CIfly	d-collision	TrekIndex	Speedup
Covariate screening									
CauseNet	100,587	2,513 s	1,214 s	3,837 s	4,043 s	1,468 s	3,643 s	**259 ms**	4,696
ASER	109,781	2,166 s	796 s	4,048 s	4,145 s	980 s	2,947 s	**1.76 s**	453
Literature replay									
CauseNet	164,568	12,590 s	2,953 s	1,661 s	3,811 s	8,018 s	24,560 s	**1.15 s**	1,439
Conditional-independence tests of PC									
BA 500k	10,000	269 s	69.8 s	67.4 ms	315 ms	168 s	111 s	**16.6 ms**	4.06
ASER	10,000	457 s	216 s	52.7 ms	290 ms	224 s	273 s	**37.2 ms**	1.42
iKraph	10,000	140 s	41.0 s	93.0 ms	609 ms	86.8 s	72.5 s	**19.0 ms**	4.90
BA 2M	10,000	919 s	362 s	103 ms	390 ms	707 s	446 s	**23.2 ms**	4.43
CNFDAG	10,000	1,491 s	970 s	493 ms	3.46 s	883 s	979 s	**17.6 ms**	28.0
CauseNet	10,000	728 s	1,415 s	43.5 ms	95.5 ms	591 s	1,220 s	**29.1 ms**	1.50

search. On graphs with a large periphery, the projections decide most tests in every configuration, at least 81% on CauseNet and 58% on CNFDAG. On the BA graphs, which have no periphery, labels decide most tests. The share they refute grows with $|Z|$, e.g., from 51% at $|Z| = 1$ to 94% at $|Z| = 32$ on BA 2M with ancestor Z, since a larger $\mathcal{H}_Z$ contains more pairs of vertices that no trek connects. With random Z, searches decide at most 7% of the tests on every graph. Since $\mathcal{H}_Z$ grows with $|Z|$, the searches per query nevertheless increase, up to 1.83 (BA 500k, ancestor, $|Z| = 32$), which explains the smaller speedups at large $|Z|$ in Exp-1.

7.4 Applications

► **Exp-8: Application workloads.** Table 7 reports the total query time on three workloads. *Covariate screening* runs on CauseNet and ASER, the largest and the densest of the real graphs. For random (cause, effect) pairs, it checks whether the observed parents of the cause, or of cause and effect, satisfy the back-door criterion. Every vertex is observed with probability 30%, and Z has at most 32 vertices. *Literature replay* checks the three back-door conditions of the mediation hypotheses from a study by Toonsi et al. [44]. These need a graph whose vertices match their variables by name, so we use CauseNet. The covariates of a hypothesis are the parents of its vertices, restricted to the diseases and drugs of the study. The third workload consists of the tests that the PC algorithm [8, 39] issues with conditioning sets of at most one vertex, i.e., up to level 1, when it learns the skeleton over an ancestral set from a d-separation oracle. On each graph, we take 20 ancestral sets of 800 to 1,000 vertices and sample 10,000 of their 12.8 to 20.0 million tests uniformly.

On screening, the descendant checks of the back-door criterion take most of the time of the traversals, and TrekIndex is 4,696× and 453× faster than the fastest baseline on CauseNet and ASER. On the replay, TrekIndex is 1,439× faster. On the PC tests, Anc-BB is the fastest baseline on every graph, since a test stays within an ancestral set of at most 1,000 vertices; TrekIndex is 1.42× (ASER) to 28.0× (CNFDAG) faster. BB, CIfly, Geiger, and d-collision are at least 2,161× slower than TrekIndex.

8 RELATED WORK

► **Causal effect identification.** Identifying causal effects from observational data given a causal DAG has been studied extensively, and the do-calculus [24] is complete for this task [13, 37]. Beyond the criteria of Section 2, prior work characterizes all valid adjustment sets [30, 38], finds minimal adjustment sets and separators efficiently [46, 47], finds and lists front-door sets in polynomial time [14, 48], and gives graphical identification conditions [42] and automated searches over the rules of the do-calculus [43]. These works build on d-separation. Causal analysis libraries [11, 36, 41, 57] evaluate it by a graph search and invoke it many times, as does the PC algorithm [8, 39] run against a known DAG. Our work is complementary to these works: instead of proposing new criteria or algorithms, we provide one index that answers their d-separation and directed-path tests on large DAGs.

► **Causality in data management.** Causality has been applied to query explanation [19, 20, 32], bias removal in OLAP queries [33], explanation of confounding bias [52], causal data integration [53], and what-if analysis [9, 22]. For graph data, Pachera et al. [22] envision causally aware graph databases, in which a property graph embeds the causal DAG and declarative queries express interventions and counterfactuals. The d-collision graph method [23] computes d-separation in an existing graph database with openCypher queries or user-defined functions, deriving a new graph for each conditioning set. None of these works, however, indexes the causal DAG on which their analyses rely. Closest to our work in data management, causal DAG summarization [54] reduces a causal DAG for downstream analysis, whereas witness contraction reduces the trek DAG inside an index and preserves exact answers.

9 CONCLUSION

In this paper, we presented TrekIndex, an index for d-separation on large causal DAGs that also answers the criteria built on it. It reduces every query to forbidden-set reachability tests on a trek DAG that does not depend on the query, and its peripheral data and the 2-hop labels on the core left by witness contraction decide most of these tests without a search. On six datasets with up to 12.2 million vertices, TrekIndex answers d-separation queries 4.84× to 6,935× faster than the fastest of six traversal baselines, and queries of the criteria 69.6× to 5,552× faster. In future work, we plan to maintain the index under edge updates from causal discovery and to support mixed graphs with latent confounders and criteria that ignore individual edges, such as complete adjustment criteria.

REFERENCES

[1] Takuya Akiba, Yoichi Iwata, and Yuichi Yoshida. 2013. Fast Exact Shortest-Path Distance Queries on Large Networks by Pruned Landmark Labeling. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 349–360. https://doi.org/10.1145/2463676.2465315

[2] Albert-László Barabási and Réka Albert. 1999. Emergence of Scaling in Random Networks. *Science* 286, 5439 (1999), 509–512. https://doi.org/10.1126/science.286.5439.509

[3] Elias Bareinboim, Jin Tian, and Judea Pearl. 2014. Recovering from Selection Bias in Causal and Statistical Inference. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2410–2416. https://doi.org/10.1609/aaai.v28i1.9074

[4] Surender Baswana, Keerti Choudhary, and Liam Roditty. 2016. Fault Tolerant Subgraph for Single Source Reachability: Generic and Optimal. In *Proceedings of the ACM Symposium on Theory of Computing (STOC)*. 509–518. https://doi.org/10.1145/2897518.2897648

[5] Cory J. Butz, André E. dos Santos, and Jhonatan S. Oliveira. 2016. Relevant Path Separation: A Faster Method for Testing Independencies in Bayesian Networks. In *Proceedings of the International Conference on Probabilistic Graphical Models (PGM) (Proceedings of Machine Learning Research)*, Vol. 52. 74–85.

[6] Keerti Choudhary. 2016. An Optimal Dual Fault Tolerant Reachability Oracle. In *Proceedings of the International Colloquium on Automata, Languages, and Programming (ICALP) (Leibniz International Proceedings in Informatics (LIPIcs))*, Vol. 55. 130:1–130:13. https://doi.org/10.4230/LIPIcs.ICALP.2016.130

[7] Edith Cohen, Eran Halperin, Haim Kaplan, and Uri Zwick. 2003. Reachability and Distance Queries via 2-Hop Labels. *SIAM J. Comput.* 32, 5 (2003), 1338–1355. https://doi.org/10.1137/S0097539702403098

[8] Diego Colombo and Marloes H. Maathuis. 2014. Order-Independent Constraint-Based Causal Structure Learning. *Journal of Machine Learning Research* 15, 116 (2014), 3921–3962.

[9] Sainyam Galhotra, Amir Gilad, Sudeepa Roy, and Babak Salimi. 2022. HypeR: Hypothetical Reasoning With What-If and How-To Queries Using a Probabilistic Causal Approach. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1598–1611. https://doi.org/10.1145/3514221.3526149

[10] Dan Geiger, Thomas Verma, and Judea Pearl. 1990. Identifying Independence in Bayesian Networks. *Networks* 20, 5 (1990), 507–534. https://doi.org/10.1002/net.3230200504

[11] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the Python in Science Conference (SciPy)*. 11–15.

[12] Stefan Heindorf, Yan Scholten, Henning Wachsmuth, Axel-Cyrille Ngonga Ngomo, and Martin Potthast. 2020. CauseNet: Towards a Causality Graph Extracted from the Web. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*. 3023–3030. https://doi.org/10.1145/3340531.3412763

[13] Yimin Huang and Marco Valtorta. 2006. Pearl's Calculus of Intervention Is Complete. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*. 217–224.

[14] Hyunchai Jeong, Jin Tian, and Elias Bareinboim. 2022. Finding and Listing Front-door Adjustment Sets. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 33173–33185. https://doi.org/10.52202/068431-2404

[15] Ruoming Jin, Hui Hong, Haixun Wang, Ning Ruan, and Yang Xiang. 2010. Computing Label-Constraint Reachability in Graph Databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 123–134. https://doi.org/10.1145/1807167.1807183

[16] Ruoming Jin and Guan Wang. 2013. Simple, Fast, and Scalable Reachability Oracle. *Proceedings of the VLDB Endowment* 6, 14 (2013), 1978–1989. https://doi.org/10.14778/2556549.2556578

[17] Steffen L. Lauritzen, A. Philip Dawid, Birgitte N. Larsen, and Hans-Georg Leimer. 1990. Independence Properties of Directed Markov Fields. *Networks* 20, 5 (1990), 491–505. https://doi.org/10.1002/net.3230200503

[18] Wentao Li, Miao Qiao, Lu Qin, Ying Zhang, Lijun Chang, and Xuemin Lin. 2020. Scaling Up Distance Labeling on Graphs with Core-Periphery Properties. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1367–1381. https://doi.org/10.1145/3318464.3389748

[19] Pingchuan Ma, Rui Ding, Shuai Wang, Shi Han, and Dongmei Zhang. 2023. XInsight: eXplainable Data Analysis Through The Lens of Causality. *Proceedings of the ACM on Management of Data* 1, 2, Article 156 (2023), 27 pages. https://doi.org/10.1145/3589301

[20] Alexandra Meliou, Wolfgang Gatterbauer, Joseph Y. Halpern, Christoph Koch, Katherine F. Moore, and Dan Suciu. 2010. Causality in Databases. *IEEE Data Engineering Bulletin* 33, 3 (2010), 59–67.

[21] Dian Ouyang, Lu Qin, Lijun Chang, Xuemin Lin, Ying Zhang, and Qing Zhu. 2018. When Hierarchy Meets 2-Hop-Labeling: Efficient Shortest Distance Queries on Road Networks. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 709–724. https://doi.org/10.1145/3183713.3196913

[22] Amedeo Pachera, Mattia Palmiotto, Angela Bonifati, and Andrea Mauri. 2025. What If: Causal Analysis with Graph Databases. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4009–4016. https://doi.org/10.14778/3749646.3749671

[23] Mattia Palmiotto, Angela Bonifati, and Andrea Mauri. 2026. Computing d-Separation on Causal Property Graphs. *Proceedings of the ACM on Management of Data* 4, 6, Article 327 (2026), 27 pages. https://doi.org/10.1145/3856322

[24] Judea Pearl. 1995. Causal Diagrams for Empirical Research. *Biometrika* 82, 4 (1995), 669–688. https://doi.org/10.1093/biomet/82.4.669

[25] Judea Pearl. 2001. Direct and Indirect Effects. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*. 411–420.

[26] Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511803161

[27] Judea Pearl. 2014. Interpretation and Identification of Causal Mediation. *Psychological Methods* 19, 4 (2014), 459–481. https://doi.org/10.1037/a0036434

[28] Judea Pearl and James M. Robins. 1995. Probabilistic Evaluation of Sequential Plans from Causal Models with Hidden Variables. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*. 444–453.

[29] You Peng, Ying Zhang, Xuemin Lin, Lu Qin, and Wenjie Zhang. 2020. Answering Billion-Scale Label-Constrained Reachability Queries within Microsecond. *Proceedings of the VLDB Endowment* 13, 6 (2020), 812–825. https://doi.org/10.14778/3380750.3380753

[30] Emilija Perković, Johannes Textor, Markus Kalisch, and Marloes H. Maathuis. 2018. Complete Graphical Characterization and Construction of Adjustment Sets in Markov Equivalence Classes of Ancestral Graphs. *Journal of Machine Learning Research* 18, 220 (2018), 1–62.

[31] Thomas S. Richardson, Robin J. Evans, James M. Robins, and Ilya Shpitser. 2023. Nested Markov Properties for Acyclic Directed Mixed Graphs. *The Annals of Statistics* 51, 1 (2023), 334–361. https://doi.org/10.1214/22-AOS2253

[32] Sudeepa Roy, Laurel J. Orr, and Dan Suciu. 2015. Explaining Query Answers with Explanation-Ready Databases. *Proceedings of the VLDB Endowment* 9, 4 (2015), 348–359. https://doi.org/10.14778/2856318.2856329

[33] Babak Salimi, Johannes Gehrke, and Dan Suciu. 2018. Bias in OLAP Queries: Detection, Explanation, and Removal. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1021–1035. https://doi.org/10.1145/3183713.3196914

[34] Babak Salimi, Harsh Parikh, Moe Kayali, Lise Getoor, Sudeepa Roy, and Dan Suciu. 2020. Causal Relational Learning. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 241–256. https://doi.org/10.1145/3318464.3389759

[35] Ross D. Shachter. 1998. Bayes-Ball: The Rational Pastime (for Determining Irrelevance and Requisite Information in Belief Networks and Influence Diagrams). In *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*. 480–487.

[36] Amit Sharma and Emre Kiciman. 2020. DoWhy: An End-to-End Library for Causal Inference. arXiv:2011.04216

[37] Ilya Shpitser and Judea Pearl. 2006. Identification of Joint Interventional Distributions in Recursive Semi-Markovian Causal Models. In *Proceedings of the National Conference on Artificial Intelligence (AAAI)*. 1219–1226.

[38] Ilya Shpitser, Tyler VanderWeele, and James M. Robins. 2010. On the Validity of Covariate Adjustment for Estimating Causal Effects. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*. 527–536.

[39] Peter Spirtes, Clark Glymour, and Richard Scheines. 2000. *Causation, Prediction, and Search* (2nd ed.). MIT Press.

[40] Seth Sullivant, Kelli Talaska, and Jan Draisma. 2010. Trek Separation for Gaussian Graphical Models. *The Annals of Statistics* 38, 3 (2010), 1665–1685. https://doi.org/10.1214/09-AOS760

[41] Johannes Textor, Benito van der Zander, Mark S. Gilthorpe, Maciej Liśkiewicz, and George T. H. Ellison. 2016. Robust Causal Inference Using Directed Acyclic Graphs: The R Package 'dagitty'. *International Journal of Epidemiology* 45, 6 (2016), 1887–1894. https://doi.org/10.1093/ije/dyw341

[42] Jin Tian and Judea Pearl. 2002. A General Identification Condition for Causal Effects. In *Proceedings of the National Conference on Artificial Intelligence (AAAI)*. 567–573.

[43] Santtu Tikka, Antti Hyttinen, and Juha Karvanen. 2021. Causal Effect Identification from Multiple Incomplete Data Sources: A General Search-Based Approach. *Journal of Statistical Software* 99, 5 (2021), 1–40. https://doi.org/10.18637/jss.v099.i05

[44] Sumyyah Toonsi, Paul N. Schofield, and Robert Hoehndorf. 2026. Causal Knowledge Graph Analysis Identifies Adverse Drug Effects. *Bioinformatics* 42, 1, Article btaf661 (2026), 9 pages. https://doi.org/10.1093/bioinformatics/btaf661

[45] Lucien D. J. Valstar, George H. L. Fletcher, and Yuichi Yoshida. 2017. Landmark Indexing for Evaluation of Label-Constrained Reachability Queries. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 345–358. https://doi.org/10.1145/3035918.3035955

[46] Benito van der Zander and Maciej Liśkiewicz. 2019. Finding Minimal d-separators in Linear Time and Applications. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI) (Proceedings of Machine Learning Research)*, Vol. 115. 637–647.

[47] Benito van der Zander, Maciej Liśkiewicz, and Johannes Textor. 2019. Separators and Adjustment Sets in Causal Graphs: Complete Criteria and an Algorithmic Framework. *Artificial Intelligence* 270 (2019), 1–40. https://doi.org/10.1016/j.artint.2018.12.006

[48] Marcel Wienöbst, Benito van der Zander, and Maciej Liśkiewicz. 2024. Linear-Time Algorithms for Front-Door Adjustment in Causal Graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 20577–20584. https://doi.org/10.1609/aaai.v38i18.30043

[49] Marcel Wienöbst, Sebastian Weichwald, and Leonard Henckel. 2025. Linear-Time Primitives for Algorithm Development in Graphical Causal Inference. arXiv:2506.15758

[50] Yosuke Yano, Takuya Akiba, Yoichi Iwata, and Yuichi Yoshida. 2013. Fast and Scalable Reachability Queries on Graphs by Pruned Labeling with Landmarks and Paths. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*. 1601–1606. https://doi.org/10.1145/2505515.2505724

[51] Hilmi Yıldırım, Vineet Chaoji, and Mohammed J. Zaki. 2010. GRAIL: Scalable Reachability Index for Large Graphs. *Proceedings of the VLDB Endowment* 3, 1 (2010), 276–284. https://doi.org/10.14778/1920841.1920879

[52] Brit Youngmann, Michael J. Cafarella, Yuval Moskovitch, and Babak Salimi. 2023. On Explaining Confounding Bias. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 1846–1859. https://doi.org/10.1109/ICDE55515.2023.00144

[53] Brit Youngmann, Michael J. Cafarella, Babak Salimi, and Anna Zeng. 2023. Causal Data Integration. *Proceedings of the VLDB Endowment* 16, 10 (2023), 2659–2665. https://doi.org/10.14778/3603581.3603602

[54] Anna Zeng, Michael J. Cafarella, Batya Kenig, Markos Markakis, Brit Youngmann, and Babak Salimi. 2025. Causal DAG Summarization. *Proceedings of the VLDB Endowment* 18, 6 (2025), 1933–1947. https://doi.org/10.14778/3725688.3725717

[55] Hongming Zhang, Xin Liu, Haojie Pan, Yangqiu Song, and Cane Wing-Ki Leung. 2020. ASER: A Large-scale Eventuality Knowledge Graph. In *Proceedings of The Web Conference (WWW)*. 201–211. https://doi.org/10.1145/3366423.3380107

[56] Yuan Zhang, Xin Sui, Feng Pan, Kaixian Yu, Keqiao Li, Shubo Tian, Arslan Erdengasileng, Qing Han, Wanjing Wang, Jianan Wang, Jian Wang, Donghu Sun, Henry Chung, Jun Zhou, Eric Zhou, Ben Lee, Peili Zhang, Xing Qiu, Tingting Zhao, and Jinfeng Zhang. 2025. A Comprehensive Large-Scale Biomedical Knowledge Graph for AI-Powered Data-Driven Biomedical Research. *Nature Machine Intelligence* 7, 4 (2025), 602–614. https://doi.org/10.1038/s42256-025-01014-w

[57] Yujia Zheng, Biwei Huang, Wei Chen, Joseph Ramsey, Mingming Gong, Ruichu Cai, Shohei Shimizu, Peter Spirtes, and Kun Zhang. 2024. Causal-learn: Causal Discovery in Python. *Journal of Machine Learning Research* 25, 60 (2024), 1–8.

A PROOFS

A.1 Proofs for Section 2

We show that Table 2 decides the back-door criterion and the recoverability condition of selection bias exactly.

Back-door criterion. Table 2 tests the blocking condition as $(X \perp\!\!\!\perp Y \mid Z)_{\underline{X}}$, which is equivalent when Z contains no descendant of X. In $\mathcal{G}_{\underline{X}}$, the paths from X to Y that meet X only at their first vertex are exactly the back-door paths. Every other path from X to Y passes through a second vertex of X, a collider without children that blocks it. Since no vertex of Z is a descendant of X, ignoring the edges out of X changes no ancestor of Z, so these paths are blocked in $\mathcal{G}$ and in $\mathcal{G}_{\underline{X}}$ alike.

Selection bias. Table 2 tests $(Y \perp\!\!\!\perp S \mid X)$ on $\mathcal{G}$ as $P \cap Y = \emptyset$ and $(Y \perp\!\!\!\perp P \setminus X \mid X)$. Every path from Y to S ends with an edge $p \to S$, $p \in P$. If $p \in Y$, this edge alone is an open path. Otherwise, p is a non-collider of the path. The path is then blocked if $p \in X$, and, if $p \notin X$, exactly when its part from Y to p is blocked. Conversely, every path of $\mathcal{G}$ from Y to p extends to S by this edge. Since S has no children, adding it changes no ancestor of X, so a path of $\mathcal{G}$ is blocked by X in both graphs alike. Hence, the condition holds if and only if $P \cap Y = \emptyset$ and $(Y \perp\!\!\!\perp P \setminus X \mid X)$.

A.2 Proofs for Section 4

We give the complete proofs of Theorem 4.3 and Lemmas 4.4 and 4.5.

Proof of Theorem 4.3. By Step 1, the paths of $\mathcal{T}$ from $start(a)$ to $end(b)$ correspond to treks from a to b. If $start(a) = P_a$, they correspond exactly to the treks that leave a for a parent, and if $end(b) = E_b$, to those that enter b from a parent. The inner states of such a path are the states of the inner vertices of its trek, together with a state of a if the path starts at U_a and a state of b if it ends at D_b. In these two cases, a or b is in $X \cup Y$ and hence not in Z. Thus, the inner states avoid $\mathcal{S}(Z)$ if and only if no inner vertex of the trek is in Z.

Suppose first that $x \in X$ and $y \in Y$ are d-connected given Z, and let ρ be a walk between them whose colliders are in Z and whose non-colliders are not. Let $c_1, \dots, c_r$ be the colliders of ρ in order, $c_0 = x$, and $c_{r+1} = y$. The part of ρ from c_i to c_{i+1} has no collider in between, so it is a trek, and its inner vertices are non-colliders of ρ and thus not in Z. Both edges of ρ at c_i point into c_i, so the trek before c_i enters it from a parent and the trek after c_i leaves it for a parent. Hence, $start(c_i) \rightsquigarrow_{\mathcal{S}(Z)} end(c_{i+1})$ for $0 \le i \le r$. Consecutive colliders may be the same vertex, which a walk may visit twice. Merging such repetitions, $c_0, \dots, c_{r+1}$ is a walk of $\mathcal{H}_Z$, so $\mathcal{H}_Z$ has a path from x to y.

Conversely, let $a_0, \dots, a_q$ be a shortest path of $\mathcal{H}_Z$ from a vertex of X to a vertex of Y. By minimality, $a_1, \dots, a_{q-1}$ are not in $X \cup Y$, so they are in Z. Since $\mathcal{H}_Z$ is undirected, each edge gives $start(a_i) \rightsquigarrow_{\mathcal{S}(Z)} end(a_{i+1})$ and thus a trek T_i from a_i to a_{i+1} whose inner vertices are not in Z. Concatenating $T_0, \dots, T_{q-1}$ gives a walk ρ. An inner position of some T_i is a non-collider of ρ, since a trek has no collider, and its vertex is not in Z. At a_i with $0 < i < q$, T_{i-1} enters a_i from a parent and T_i leaves it for a parent, so a_i is a collider of ρ and is in Z. By the walk characterization, a_0 and a_q are d-connected given Z. □

Proof of Lemma 4.4. By Definitions 4.1 and 2.2, $\mathcal{T}_{\overline{A}\underline{C}}$ is $\mathcal{T}$ without the arcs that come from ignored edges. These are exactly the arcs out of P_a and D_c and the arcs into E_a and U_c, for $a \in A$ and $c \in C$. A path of $\mathcal{T}$ is a path of $\mathcal{T}_{\overline{A}\underline{C}}$ if and only if it uses none of them. On a path, an inner state has an arc in and an arc out, the start state only an arc out, and the end state only an arc in. Hence, the path uses an excluded arc if and only if an inner state is in $\mathcal{B}_{A,C}$, it starts at P_a or D_c, or it ends at E_a or U_c. □

Proof of Lemma 4.5. A path of $\mathcal{T}$ that starts at a D state stays among the E and D states, so a path from D_x to E_y has the form $D_x \to E_{u_1} \to D_{u_1} \to \cdots \to E_{u_k} \to D_{u_k} \to E_y$. It corresponds to the directed path $x \to u_1 \to \cdots \to u_k \to y$ of $\mathcal{G}$, and vice versa. Its inner states are the states of $u_1, \dots, u_k$, so they avoid $\mathcal{S}(F)$ if and only if no inner vertex of the directed path is in F. □

A.3 Proofs for Section 5

We give the complete proof of Lemma 5.3 and details of the construction.

Proof of Lemma 5.3. By induction on i; the case $i = 0$ is trivial. Four facts hold in every $\mathcal{T}^{(i)}$. (F1) Every arc between states of different vertices goes from a P or D state to a U or E state. This holds for $\mathcal{T}$, and a shortcut arc takes its tail from an arc into v_i and its head from an arc out of v_i. (F2) Expanding shortcut arcs maps a path of $\mathcal{T}^{(i)}$ to a path of $\mathcal{T}$ that contains its states in the same order. In particular, $\mathcal{T}^{(i)}$ is acyclic. (F3) A path that leaves a vertex c and returns to it leaves at P_c and returns at E_c. By (F2), it suffices to show this in $\mathcal{T}$. There, a path moves only down after D_c, and after P_c it moves up through proper ancestors of c and, once it turns, never reaches an up state again. (F4) No path leads from an E or D state to a U or P state. This holds for $\mathcal{T}$, as in (F3), and by (F2), in every $\mathcal{T}^{(i)}$.

Suppose first that $\mathcal{T}^{(i)}$ has an $\mathcal{F}$-free path from s to t. Replace every arc of this path that was added for v_i by the two arcs and the states of v_i that it composes. This gives a walk of $\mathcal{T}^{(i-1)}$, which is a path since $\mathcal{T}^{(i-1)}$ is acyclic. Its new inner states lie in witnesses and thus avoid $\mathcal{F}$, and its arcs are open, since their witnesses are subsets of open witnesses. By induction, $s \rightsquigarrow_{\mathcal{F}} t$ in $\mathcal{T}$.

Conversely, suppose that $s \rightsquigarrow_{\mathcal{F}} t$ in $\mathcal{T}$. By induction, $\mathcal{T}^{(i-1)}$ has an open path from s to t whose inner states avoid $\mathcal{F}$. Let π be one with the fewest arcs. If π avoids v_i, it is a path of $\mathcal{T}^{(i)}$. Otherwise, by (F1), π enters v_i at a U or E state, moves by internal arcs, and leaves at a P or D state. If π visited v_i twice, it would contain $U_{v_i} \to P_{v_i} \rightsquigarrow E_{v_i} \to D_{v_i}$ by (F3), and the arc $U_{v_i} \to D_{v_i}$ would give a shorter path. Hence π visits v_i once, between arcs $\alpha \to \sigma$ and $\sigma' \to \beta$. Suppose that α and β belong to the same vertex c. Then $\alpha = P_c$ and $\beta = E_c$ by (F1) and (F3). By (F1), P_c is entered only from U_c, and E_c is left only to D_c. If $P_c \neq s$ and $E_c \neq t$, then π contains $U_c \to P_c \rightsquigarrow E_c \to D_c$, and the arc $U_c \to D_c$ again gives a shorter path. If $P_c = s$, then E_c and D_c are inner states of π, since t is not a state of c, which contradicts (i). If $E_c = t$, then s reaches the up state P_c and is thus a U or P state by (F4). Moreover, U_c and P_c are inner states of π, which contradicts (ii). Hence α and β belong to different vertices, and $\mathcal{T}^{(i)}$ has the shortcut arc $\alpha \to \beta$. Its witness consists of the witnesses of two open arcs and of inner

states of π, so the arc is open. Replacing the visit of v_i by this arc gives the required path of $\mathcal{T}^{(i)}$. □

Parallel arcs. Parallel arcs multiply when they are composed, so their number is not bounded in terms of w.

Complexity of the construction. Let ℓ be the maximum label size, m_w the number of arcs of $\mathcal{T}_w$, m_s the number of shortcut arcs created during elimination, and f the number of fill edges. Elimination takes $O((n + m + f) \log n)$ time, plus the examination of pairs, which is at most the product of the numbers of arcs into and out of each eliminated vertex. Pruned labeling takes $O(\ell^2 m_w)$ time, since every label entry scans the arcs of its state and every scanned arc costs one label check. The index takes $O(|K|\ell + m_w)$ space for the labels and the witness DAG, and $O(m_s)$ for the records of all shortcut arcs, since each stores its ends, the eliminated vertex with the states that it passes, and references to its two constituent arcs, so that every witness is a tree over these records. The peripheral data take $O(\sum_v(|chain(v)| + l_v + c_v))$ space, where l_v and c_v are the numbers of local arcs of v and of arcs of $\mathcal{T}_w$ that v witnesses.

A.4 Proofs for Section 6

We prove Lemmas 6.1 and 6.2 and Theorem 6.3. Besides facts (F1)–(F4) from the proof of Lemma 5.3, which hold in every $\mathcal{T}^{(i)}$, we use one more fact. (F5) Every arc of $\mathcal{T}^{(i)}$ between states of two vertices joins vertices that are adjacent in the skeleton after the first i eliminations, including fill edges. This holds for $\mathcal{T}$, and a shortcut arc added for v_i joins two portals of v_i, which the fill edges make adjacent. Hence, when v_{i+1} is eliminated, every arc of $\mathcal{T}^{(i)}$ at a state of v_{i+1} is an internal arc or a local arc of v_{i+1}.

Proof of Lemma 6.1. If s is a core state, $fwd(s) = \{s\}$. Otherwise, the forward projection expands s and every reached state not in $\mathcal{F}$. From an expanded state of a vertex u, it follows the internal arcs of u and, if u is peeled, the open local arcs of u out of this state. The backward projection is symmetric. The chain masks replace this search only when no state of the vertex of s or of its chain is forbidden and the local arcs along the chain have empty witnesses. Then every reached state is expanded and every local arc is open, so the masks give the same projection.

Suppose first that (i) or (ii) holds. Every state $u \in fwd(s)$ is reached from s along internal arcs and open local arcs whose inner states are expanded and thus not in $\mathcal{F}$. Expanding every local arc into the path of $\mathcal{T}$ that it stands for gives a walk of $\mathcal{T}$, which is an $\mathcal{F}$-free path, since $\mathcal{T}$ is acyclic and the new inner states lie in open witnesses. The same holds for $bwd(t)$ and, in case (ii), for the path of $\mathcal{T}_w$ from σ to τ. Joining these paths at u in case (i), or at σ and τ in case (ii), gives a walk of $\mathcal{T}$ from s to t, which is a path since $\mathcal{T}$ is acyclic. A joining state is allowed, so it is not in $\mathcal{F}$ if it is an inner state. Hence, $s \rightsquigarrow_{\mathcal{F}} t$.

Conversely, suppose that $s \rightsquigarrow_{\mathcal{F}} t$ and that (i) does not hold. Let k be the number of peeled vertices, so that $\mathcal{T}^{(k)} = \mathcal{T}_w$. A *bridge* at level i is a pair of states $\beta \in fwd(s)$ and $\gamma \in bwd(t)$ of two distinct vertices other than $v_1, \dots, v_i$ such that $\beta = s$ or β is a U or E state not in $\mathcal{F}$, $\gamma = t$ or γ is a P or D state not in $\mathcal{F}$, and $\mathcal{T}^{(i)}$ has an $\mathcal{F}$-free path from β to γ. The pair (s, t) is a bridge at level 0, and a bridge at level k consists of allowed core states and gives (ii). It thus suffices to turn a bridge (β, γ) at level $i < k$ into one at level $i + 1$. Let $v = v_{i+1}$, and let π be an $\mathcal{F}$-free path of $\mathcal{T}^{(i)}$ from β to γ with the fewest arcs.

The triple $(\beta, \gamma, \mathcal{F})$ is admissible. If $\beta = P_c$, then $\beta = s$, and condition (i) of the admissibility of $(s, t, \mathcal{F})$ applies. If $\gamma = E_c$ and β is a U or P state, then $\gamma = t$, and s is a U or P state by (F4), since s reaches β in $\mathcal{T}$ by the first part. Hence, condition (ii) of the admissibility of $(s, t, \mathcal{F})$ applies.

If v is the vertex of neither β nor γ, the argument for the converse direction in the proof of Lemma 5.3, applied to $(\beta, \gamma, \mathcal{F})$, gives an $\mathcal{F}$-free path of $\mathcal{T}^{(i+1)}$ from β to γ, so (β, γ) is a bridge at level $i + 1$.

Suppose that v is the vertex of β. Then s is not a core state, since otherwise $fwd(s) = \{s\}$ would give $\beta = s$, a state of the peeled vertex v. Since γ is not a state of v, π leaves v, and it does not return. Otherwise, by (F3), π contains P_v and later $E_v \to D_v$. If $\beta = P_v$, then E_v and D_v are inner states of π, which contradicts the admissibility of $(\beta, \gamma, \mathcal{F})$. If $\beta \neq P_v$, then P_v is entered from U_v by (F1), and replacing $U_v \to P_v \rightsquigarrow E_v \to D_v$ by the arc $U_v \to D_v$ gives a shorter path. Hence, π consists of at most one internal arc from β to a state β' of v, an arc from β' to a state β'' of another vertex, and a rest that avoids the states of v. By (F5), the arc $\beta' \to \beta''$ is a local arc of v, and it is open. Since β' is β or an inner state of π, the projection expands it, so $\beta'' \in fwd(s)$. As (i) does not hold, $\beta'' \neq \gamma$, so β'' is an inner state of π and not in $\mathcal{F}$. By (F1), it is a U or E state. The rest of π is an $\mathcal{F}$-free path of $\mathcal{T}^{(i+1)}$, which keeps the arcs of $\mathcal{T}^{(i)}$ that avoid the states of v. Hence, (β'', γ) is a bridge at level $i + 1$, provided that β'' and γ belong to different vertices.

Suppose that both belong to a vertex y, and let π' be an $\mathcal{F}$-free path of $\mathcal{T}^{(i+1)}$ from β'' to γ with the fewest arcs. If π' leaves y, then by (F3) it contains P_y and later E_y, and by (F1), P_y is entered from U_y, which is β'' or an inner state. If $\gamma = E_y$, then $\gamma = t$, and U_y and P_y are not in $\mathcal{F}$. This contradicts the admissibility of $(s, t, \mathcal{F})$, since s reaches P_y and is thus a U or P state by (F4). If $\gamma \neq E_y$, then π' continues by $E_y \to D_y$, and the arc $U_y \to D_y$ shortens it. Hence, π' consists of at most one internal arc of y, along which the projection expands β''. Then $\gamma \in fwd(s) \cap bwd(t)$ is allowed, so (i) holds, a contradiction. The case that v is the vertex of γ is symmetric under reversing $\mathcal{T}$, which swaps U with D and P with E and exchanges the two conditions of admissibility: where the argument above uses (i), the mirrored one uses (ii), with s a U or P state by (F4), and in the case of a common vertex y, it uses (i). □

Proof of Lemma 6.2. All paths in this proof are paths of $\mathcal{T}_w$. We use two facts. (F6) Every state of $\mathcal{T}_w$ in $\mathcal{F}$ outside $\mathcal{E}$ is P_z or D_z of a vertex z whose states are all forbidden, so by (F1), every arc into it leaves U_z or E_z, which is in $\mathcal{E}$. (F7) The state σ is not U_z or E_z for such a vertex z. If σ is not in $\mathcal{F}$, this holds since $U_z, E_z \in \mathcal{F}$. Otherwise, σ is the start state s of the test, since an allowed state of $fwd(s)$ other than s is not t by line 20 and thus not in $\mathcal{F}$, and a turning state is tested only if it is not in $\mathcal{F}$. No edge test starts at an E state, and an edge test that starts at a U state starts at the live start state U_v of a vertex $v \in X \cup Y$ outside C, which is not in $\mathcal{F}$.

We next show a claim. Let σ' be σ or a state not in $\mathcal{F}$, and let τ' be a state that σ' reaches. Suppose that no state of $\mathcal{E}$ other than σ' and τ' lies on a path from σ' to τ', and that no blocked arc leads from σ' to τ'. Then every path π from σ' to τ' is $\mathcal{F}$-free. Indeed, σ' is not U_z or E_z for a vertex z whose states are all forbidden: if $\sigma' = \sigma$, by (F7), and otherwise since $U_z, E_z \in \mathcal{F}$. If an inner state of

π is in $\mathcal{F}$, then it is in $\mathcal{E}$, or by (F6) its predecessor on π is U_z or E_z of a vertex z whose states are all forbidden; this predecessor is in $\mathcal{E}$ and is not σ', so it is an inner state of π. If π uses a blocked arc, both ends of this arc are in $\mathcal{E}$, and since no blocked arc leads from σ' to τ', one of them is an inner state of π. Both cases contradict the assumption.

For (a), since the labels form a 2-hop cover, $L_{out}(\sigma) \cap L_{in}(\tau) = \emptyset$ if and only if no path leads from σ to τ, and then no $\mathcal{F}$-free path does. For (b), by the same property, $L_{out}(\sigma) \cap L_{in}(\tau) \neq \emptyset$ gives a path from σ to τ, and $row(\sigma) \wedge col(\tau)$ consists of the states $\mu \in \mathcal{E}$ with $\sigma \rightsquigarrow \mu \rightsquigarrow \tau$. If the certificate holds, no state of $\mathcal{E}$ other than σ and τ lies on a path from σ to τ, and no blocked arc leads from σ to τ, so the claim with $\sigma' = \sigma$ and $\tau' = \tau$ shows that this path is $\mathcal{F}$-free.

For (c), apart from σ and τ, the search expands only states not in $\mathcal{F}$, and it follows only open arcs. Hence, every forward state has an $\mathcal{F}$-free path from σ, and every backward state has an $\mathcal{F}$-free path to τ.

($\Rightarrow$) Suppose that the search succeeds. If the frontiers meet, joining the two paths gives an $\mathcal{F}$-free path from σ to τ. Otherwise, the search succeeds early at a kept state u. If u is a forward state, it reaches τ, and $row(u) \wedge col(\tau)$ without τ is empty. This set holds the states of $\mathcal{E}$ other than τ that lie on a path from u to τ, including u itself if $u \in \mathcal{E}$. Hence, $u \notin \mathcal{E}$, no state of $\mathcal{E}$ other than u and τ lies on a path from u to τ, and no blocked arc leads from u to τ, since both ends of a blocked arc are in $\mathcal{E}$. By the claim with $\sigma' = u$ and $\tau' = \tau$, every path from u to τ is $\mathcal{F}$-free, and joining one with the $\mathcal{F}$-free path from σ to u gives an $\mathcal{F}$-free path from σ to τ. If u is a backward state, σ reaches it, and $row(\sigma) \wedge col(u)$ without σ is empty. As above, $u \notin \mathcal{E}$, no state of $\mathcal{E}$ other than σ and u lies on a path from σ to u, and no blocked arc leads from σ to u. By the claim with $\sigma' = \sigma$ and $\tau' = u$, every path from σ to u is $\mathcal{F}$-free, and joining one with the $\mathcal{F}$-free path from u to τ gives an $\mathcal{F}$-free path from σ to τ.

($\Leftarrow$) Suppose that an $\mathcal{F}$-free path π from σ to τ exists. Every inner state of π is not in $\mathcal{F}$, is reached from σ, and reaches τ, so the label pruning keeps it, and every arc of π is open. A frontier becomes empty only after its side has visited every state that it can reach along open arcs through kept states. The forward side can reach τ along π, and τ is visited by the backward side, so the forward frontier cannot become empty before the frontiers meet. Symmetrically, neither can the backward frontier, so the search succeeds. □

Proof of Theorem 6.3. We prove (1) and (2) in turn. For a Sep condition, $\mathcal{F} = \mathcal{S}(Z) \cup \mathcal{B}_{A,C}$, and for Reach$(x, y; F)$, $\mathcal{F} = \mathcal{S}(F)$. In both cases, the states of $\mathcal{F}$ are marked and the shortcut arcs that they witness are blocked before the edge tests, as in line 8.

Proof of (1). The states s and t of an edge test belong to distinct vertices. In a Sep test, if $s = P_c$ or $t = E_c$, then $c \in Z$ or $c \in C \setminus A$, so $U_c, D_c \in \mathcal{F}$, which gives the conditions (i) and (ii) of admissibility (Section 5.1). A Reach test starts at D_x, so neither condition applies. Hence every edge test is admissible, and $s \rightsquigarrow_{\mathcal{F}} t$ if and only if condition (i) or (ii) of Lemma 6.1 holds.

If (ii) holds but (i) does not, then (ii) also holds for a pair of dominant states. Let $\sigma \in fwd(s)$ be allowed and dominated by an allowed $U_c \in fwd(s)$, or by an allowed $E_c \in fwd(s)$ if $\sigma = D_c$. Then $\sigma \neq s$, since $fwd(s) = \{s\}$ for a core state s, and $\sigma \neq t$, since otherwise $t \in fwd(s) \cap bwd(t)$ would give (i). Hence, σ is not in $\mathcal{F}$, so prefixing an $\mathcal{F}$-free path from σ with the arc $U_c \to \sigma$ or $E_c \to \sigma$ gives an $\mathcal{F}$-free path from U_c or E_c, which are never dominated. The case of $bwd(t)$ is symmetric.

($\Rightarrow$) Suppose that EdgeTest returns true. If it returns at line 20, (i) holds. Otherwise, it returns for a pair (σ, τ) of allowed core states of distinct vertices that line 22 does not skip, either because the certificate holds, which gives an $\mathcal{F}$-free path from σ to τ by (b) of Lemma 6.2, or because the search succeeds, which gives one by (c) of Lemma 6.2. Hence (ii) holds, and $s \rightsquigarrow_{\mathcal{F}} t$.

($\Leftarrow$) Suppose that $s \rightsquigarrow_{\mathcal{F}} t$. If (i) holds, line 20 returns true. Otherwise, (ii) holds for a pair (σ, τ) of dominant states, which line 21 tries unless an earlier pair returns true. By (a) of Lemma 6.2, $L_{out}(\sigma) \cap L_{in}(\tau) \neq \emptyset$, so line 22 does not skip the pair. Then the certificate holds, or the search succeeds by (c) of Lemma 6.2. Hence EdgeTest returns true.

Both directions also hold when EdgeTest uses the up-closure of a core node with start state s. Then $fwd(s) = \{s\}$ and $bwd(t)$ contains no state of the node, so (i) fails, and (ii) asks for an $\mathcal{F}$-free path from s to an allowed core state $\tau \in bwd(t)$; the test examines every such τ. We show that such a path exists if and only if τ is in the closure, τ is a turning state, or $\mathcal{T}_w$ has an $\mathcal{F}$-free path from a turning state $d \notin \mathcal{F}$ to τ. If s is a D or E state, the closure has the single turning state s. Otherwise, by (F1) and (F4), a path from s visits U and P states, takes one arc into a D or E state, and then visits only D and E states. Hence, an $\mathcal{F}$-free path to a U or P state τ exists if and only if τ is in the closure. An $\mathcal{F}$-free path to a D or E state τ enters its first down state d by $U_r \to D_r$ or by an open shortcut arc from the closure, so d is a turning state. Moreover, $d = \tau$, or d is an inner state and thus not in $\mathcal{F}$, and the rest of the path is an $\mathcal{F}$-free path from d to τ. Conversely, each of the three cases gives an $\mathcal{F}$-free path from s to τ. For a U or P state τ, the test succeeds if and only if τ is in the closure. For a D or E state τ, it succeeds if τ is a turning state, and otherwise it runs the three layers on (d, τ) for every turning state $d \notin \mathcal{F}$, which Lemma 6.2 covers, also when d and τ belong to the same vertex. Hence both directions follow as above. The thresholds only decide whether the closure is used.

Proof of (2). Let $\mathcal{H}_Z$ be the query graph of $\mathcal{G}_{\overline{A}\underline{C}}$. By Theorem 4.3 applied to $\mathcal{G}_{\overline{A}\underline{C}}$, X and Y are d-separated by Z in $\mathcal{G}_{\overline{A}\underline{C}}$ if and only if $\mathcal{H}_Z$ has no path from S to O. By Lemma 4.4, as noted in Section 4.3, a node of $Z \cap A$ has no edge in $\mathcal{H}_Z$, and an edge between two other nodes i and j exists if and only if $start(i) \rightsquigarrow_{\mathcal{F}} end(j)$ in $\mathcal{T}$. A node of $Z \cap A$ also has no live states, so SepTest tests no edge at it.

Let $v \in X \cup Y$. If $v \in C \setminus A$, then $U_v, D_v \in \mathcal{F}$ and $P_v, E_v \notin \mathcal{F}$. A path from U_v to a state of another vertex starts with $U_v \to P_v$, since D_v cannot be an inner state, so $U_v \rightsquigarrow_{\mathcal{F}} t$ if and only if $P_v \rightsquigarrow_{\mathcal{F}} t$ for every state t of another vertex. Similarly, a path from another vertex into D_v ends with $E_v \to D_v$, so $s \rightsquigarrow_{\mathcal{F}} D_v$ if and only if $s \rightsquigarrow_{\mathcal{F}} E_v$ for every state s of another vertex. If $v \in A \setminus C$, then $P_v, E_v \in \mathcal{F}$, and the same argument shows that D_v can replace U_v as the start state and U_v can replace D_v as the end state. If $v \in A \cap C$, both successors of U_v and both predecessors of D_v are in $\mathcal{F}$, so v has no edge. For every other node, the live states are the start and end states of Definition 4.2. Hence, an edge between i and j of $\mathcal{H}_Z$ exists

if and only if $s \rightsquigarrow_{\mathcal{F}} t$ for the live start state s of i and the live end state t of j, that is, by (1), if and only if the edge test from i to j returns true.

($\Rightarrow$) By contraposition, suppose that X and Y are d-connected given Z in $\mathcal{G}_{\overline{A}\underline{C}}$, so that $\mathcal{H}_Z$ has a path $i_0, \ldots, i_k$ with $i_0 \in S$ and $i_k \in O$. Every node of this path has an edge and thus live states. Suppose, for a contradiction, that SepTest returns true. Then it pushes i_0, pops every node that it pushes, and from each popped node tests every node with live states that it has not reached. Let i_j be the first node of the path that is never reached. Then i_{j-1} is popped, and since $\mathcal{H}_Z$ is undirected, the test from i_{j-1} to i_j returns true, so i_j is reached, or SepTest returns false if $i_j \in O$, a contradiction.

($\Leftarrow$) By contraposition, suppose that SepTest returns false. Every node in R is reached from S along edges of $\mathcal{H}_Z$ whose tests returned true, and the last test reaches a node of O. Hence $\mathcal{H}_Z$ has a path from S to O, and X and Y are d-connected given Z in $\mathcal{G}_{\overline{A}\underline{C}}$.

Finally, Query returns true if and only if no edge test of a Reach condition returns true and SepTest returns true for every Sep condition; for selection bias, $P \cap Y = \emptyset$ is checked directly. By (1) and Lemma 4.5, every Reach test decides exactly whether $x \rightsquigarrow_F y$ in $\mathcal{G}$, which also gives Z' for rule 3, and by (2), every Sep condition is decided exactly. Hence every query of Table 2 is answered exactly. □

Size thresholds of the up-closure. The closure is attempted only if $\mathcal{H}_Z$ has $h \geq 6$ nodes and at least half of the node's first 16 parents are forbidden. It is dropped beyond $8 + 4h$ states or four turning states.

B ADDITIONAL RESULTS FOR SECTION 7

B.1 Setup

To keep an acyclic subgraph of a cyclic graph without reversing any edge, we keep, inside each strongly connected component, the edges of the vertex-ID order, ascending or descending, that retains more of them. For iKraph, we instead remove the edges with the least supporting evidence until no cycle remains. Besides random and ancestor sets, back-door queries also take as Z up to k parents of x, and front-door queries take as M up to k children of x. The variant of Exp-5 builds 2-hop labels on the full trek DAG and processes queries as in Section 6, except for the up-closure of walled-in nodes.

B.2 d-Separation

Figure 9 shows the percentage of queries answered d-separated. With random Z, it falls from $|Z| = 1$ to $|Z| = 32$ on every graph, e.g., from 28% to 4% on ASER: a random vertex rarely blocks a path between the nearby x and y, but it may open one as a descendant of a collider. With ancestor Z, it rises on every graph, e.g., from 38% to 64% on BA 500k, since the ancestors of x and y lie on many of the treks between them and block these treks. The speedup of TrekIndex shrinks with $|Z|$ for both kinds of sets, so it does not depend on whether most answers are d-separated or d-connected.

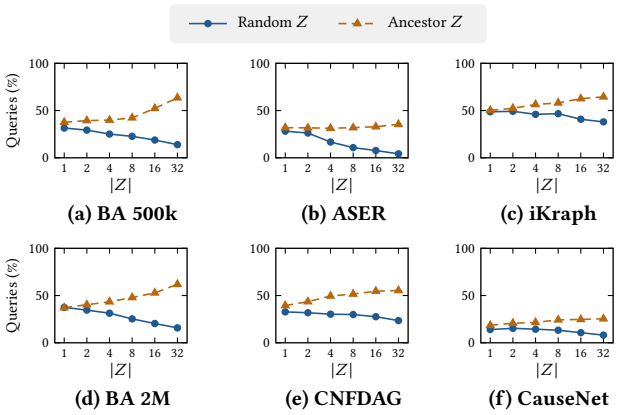


Figure 9: Percentage of queries answered d-separated in each configuration of Figure 5.

B.3 Causal Criteria

Figure 6 reports random sets of 1, 4, and 16 vertices, Figure 10 ancestor sets of the same sizes, and Figure 11 the back-door criterion with parent sets and the front-door criterion with child sets. TrekIndex is faster than every baseline in all 324 configurations. BB is the fastest baseline in 254 of them, Anc-BB in 68, and CIfly in two. The smallest margin of TrekIndex shrinks as the sets grow, from 55.1× (rule 3, BA 500k, ancestor) for one vertex to 37.5× (rule 1, BA 500k, ancestor) for four and 13.1× (selection, BA 500k, ancestor) for 16. Ancestor sets, which lie close to x and y, give a smaller margin than random sets of the same size in 103 of the 144 combinations of a criterion, a graph, and a size. With parent sets, the back-door criterion is 28.6× to 4,071× faster than the fastest baseline, and with child sets, the front-door criterion is 42.1× to 1,887× faster.

Figure 12 gives the percentage of queries for which each criterion holds. It ranges from 2% (mediation, ASER, random-16) to 98% (rule 3, BA 2M, random-1). With random sets, it falls as the sets grow for every criterion and graph, since a larger random set more often contains a descendant of a treatment or opens a path through a collider. With parent sets, the back-door criterion holds more often as the sets grow on every graph, since more parents of x block more back-door paths, whereas with child sets, the front-door criterion holds for 25% to 56% of the queries.

B.4 Witness Contraction

Table 8 compares the construction of TrekIndex with that of the variant of Exp-5, which builds 2-hop labels on the full trek DAG. On the four graphs with a periphery, witness contraction shortens the construction by 1.17× (ASER) to 7.48× (CauseNet) and makes the index 1.28× (ASER) to 8.18× (CauseNet) smaller. On the BA graphs, nothing is peeled, so the two indexes have the same labels, and their build times and sizes differ by at most 3%. On the graphs with $w = 3$, elimination creates 940 k to 1.00 M shortcut arcs, at most 0.182 per peeled vertex, and at most 934 of them (CauseNet) join the same two states.

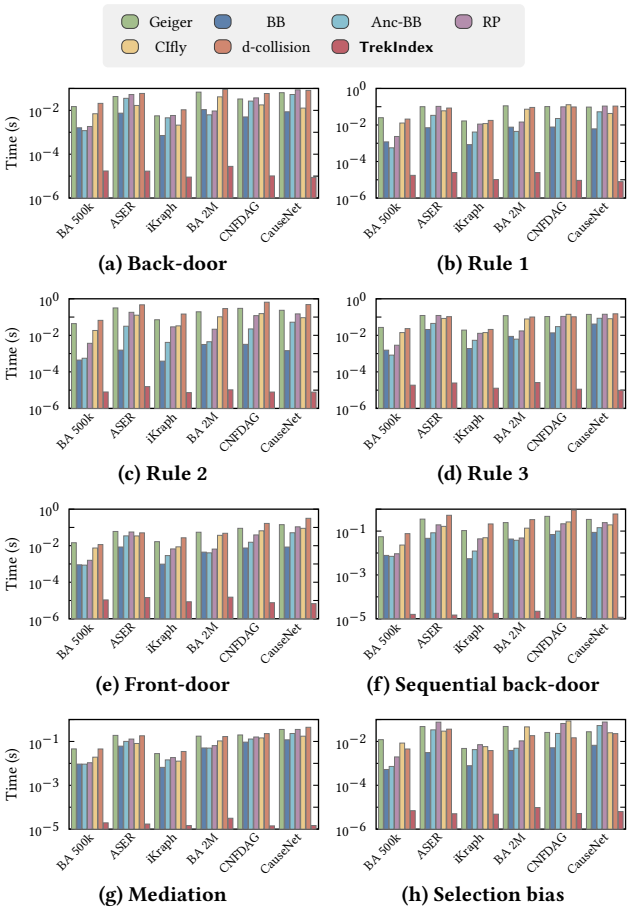


Figure 10: Mean query time of the causal criteria for ancestor sets of 1, 4, and 16 vertices.

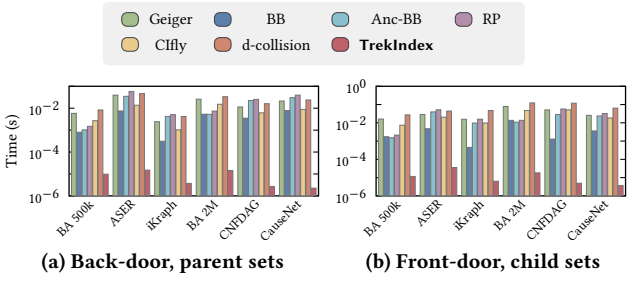


Figure 11: Mean query time of the back-door criterion with parent sets and of the front-door criterion with child sets of 1, 4, and 16 vertices.

B.5 Surgery-First Exploration

Table 9 repeats rule 2 from Table 6 and adds rule 3. On rule 3, surgery-first speeds up queries with random sets by 1.03× to 4.15×, and the gain grows with the size of the set on every graph. With ancestor sets, the gain is at most 1.21×, and surgery-first is slightly slower in 2 configurations, at worst 0.985× (CauseNet, ancestor-1; CNFDAG, ancestor-1). The two rules thus behave in opposite ways: ancestor sets gain more than random sets of the same size on rule 2, and random sets gain more on rule 3, on every graph and for every size.

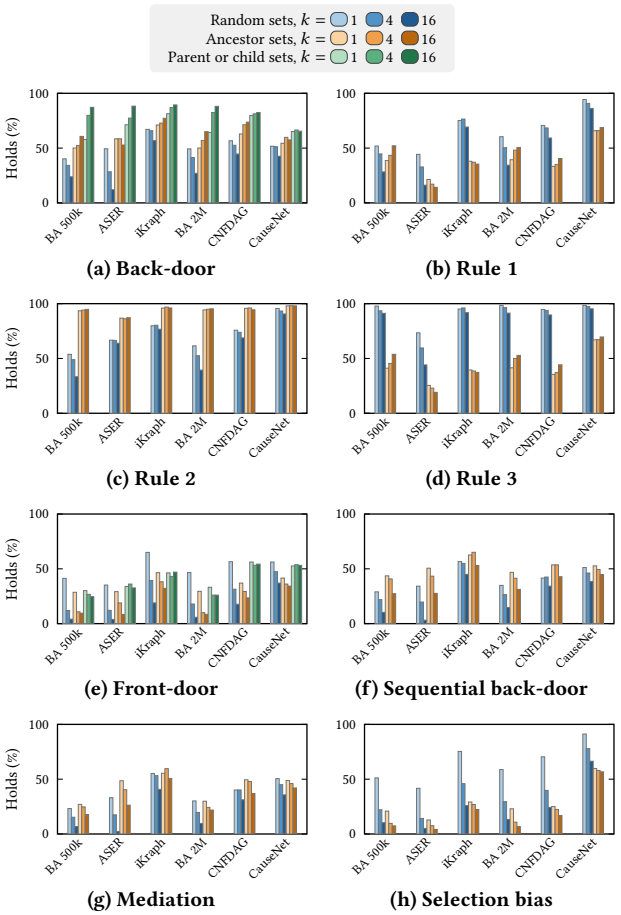


Figure 12: Percentage of queries for which each criterion holds.

Table 8: Construction of 2-hop labels on the full trek DAG (Plain) and of TrekIndex.

	Build time		Label entries		Index size	
Graph	Plain	TrekIndex	Plain	TrekIndex	Plain	TrekIndex
BA 500k	45.7 s	46.7 s	116 M	116 M	479 MiB	481 MiB
ASER	72.6 s	62.0 s	176 M	139 M	773 MiB	606 MiB
iKraph	69.4 s	43.7 s	217 M	106 M	975 MiB	496 MiB
BA 2M	368 s	371 s	681 M	681 M	2.69 GiB	2.69 GiB
CNFDAG	94.8 s	21.8 s	314 M	21.4 M	1.57 GiB	286 MiB
CauseNet	564 s	75.4 s	1,336 M	99.7 M	5.89 GiB	736 MiB

Table 9: Speedup from surgery-first exploration on rules 2 and 3.

	Random			Ancestor		
Graph	$k = 1$	$k = 4$	$k = 16$	$k = 1$	$k = 4$	$k = 16$
Rule 2						
BA 500k	1.05	1.13	1.18	1.40	2.46	3.36
ASER	1.11	1.79	2.64	1.82	6.36	7.43
iKraph	1.09	1.43	1.81	1.42	2.38	2.70
BA 2M	1.01	1.15	1.18	1.37	2.49	4.37
CNFDAG	1.08	1.48	1.80	1.27	1.53	2.28
CauseNet	1.04	1.11	1.28	1.12	1.41	1.78
Rule 3						
BA 500k	2.24	2.84	3.75	1.04	1.09	1.11
ASER	1.30	1.76	1.88	1.02	1.05	1.21
iKraph	1.23	1.79	2.10	1.01	1.02	1.04
BA 2M	2.43	3.49	4.15	1.03	1.12	1.11
CNFDAG	1.25	1.48	2.01	0.997	1.06	1.15
CauseNet	1.03	1.21	1.54	0.985	1.01	1.05